SECTION 1: COVERPAGE



UNIVERSITY OF NAIROBI

FACULTY OF ENGINEERING

DEPARTMENT OF ELECTRICAL AND INFORMATION ENGINEERING

FINAL YEAR PROJECT REPORT

PROJECT INDEX: 114

ECONOMIC FEASIBILITY STUDY OF A GIGABIT- PASSIVE OPTICAL NETWORK (GPON) BY IMPLEMENTATION OF A BILLING SYSTEM

STUDENT NAME: MUSYOKI BRIAN MUMO

REGISTRATION NUMBER: F17/142583/2021

SUPERVISOR(S): DR.P.O.AKUON

EXAMINER(S): DR.G.OMONDI

This project report is submitted in partial fulfilment of the requirement for the award of the Degree of Bachelor of Science in Electrical and Electronics Engineering from the University of Nairobi

Submitted on Monday 4th of May 2026

-

SECTION 2: OUTLINE

TITLE: AUTOMATED BILLING SYSTEM WITH MPESA STK INTERGRATION

SECTION 1: OUTLINE 1

SECTION 2: BACKGROUND 2

SECTION 3: LITERATURE REVIEW 7

SECTION 4: GAP / PROBLEM SOLVED 8

SECTION 5: DESIGN APPROACH 10

SECTION 6: SELECTED OPTION / REASONS FOR APPROACH 11

SECTION 7: METHODOLOGY 12

SECTION 8: PROGRAMME DESIGN 13

SECTION 9: BILL OF MATERIALS 14

SECTION 10: IMPLEMENTATION 15

SECTION 11: FINANCIAL RESULTS AND DISCUSSION .....

SECTION 12: CHALLENGES AND LESSONS .....

SECTION 13: FUTURE RECOMMENDATIONS ON CHALLENGES .....

SECTION 14: REFERENCES .....

SECTION 15: APPENDIX .....

SECTION 3: BACKGROUND

In an ever-changing technological environment as Kenya the need for a low cost implementation of a " *access to information* " interface is becoming more and more lucrative since for a country with a very large contrast in wealth distribution the only equalizer is the access to information , a country that can learn together grows together , and that was the main aim of the project ; to creating a cost effective , scalable and affordable public WIFI / GPON system for all and thus the concepts used in the design implementation and execution of the project or proposed business venture is ultimately meant to build towards that.

This project in a nutshell represents the design, implementation, and the evaluation of an Automated
Gigabit Passive Optical Network (GPON) Billing system with *Safaricom ™ M-Pesa STK push prompt* through the integration of a standalone already scalable payment stack system as *PAYHERO™*

So as to handle mobile money processing across cross-platform payment options, i.e bank accounts, Airtel Money and Paypal and thereafter providing a real-time network access control that is both reliable and secure all concepts that will be explained further on.

SECTION 4: LITERATURE REVIEW

One of the most important features that govern the system to be implemented on GPON is the scalability, cost effectiveness, and affordability of GPON and this brought by is its "*Broadband* "nature.

The starting point for the literature review was understanding what makes GPON worth billing for in the first place. The term broadband, in the simplest sense, describes any network that can carry multiple data streams simultaneously at high speed. That property is exactly what makes GPON economically attractive: a single fiber trunk feeding through a passive splitter can serve dozens of premises at once, with no powered equipment needed between the exchange and the customer. The ITU-T G.984[2] standard family defines how this works in practice — the Optical Line Terminal at the exchange end divides bandwidth among connected Optical Network Terminals using a Time Division Multiple Access scheme going upstream, while broadcasting to all terminals simultaneously going downstream. The practical consequence of this architecture is that adding new subscribers to an existing GPON deployment is far cheaper than laying dedicated copper or fiber to each individual premises, which makes it a genuinely feasible technology for bringing affordable connectivity to communities that have historically been underserved

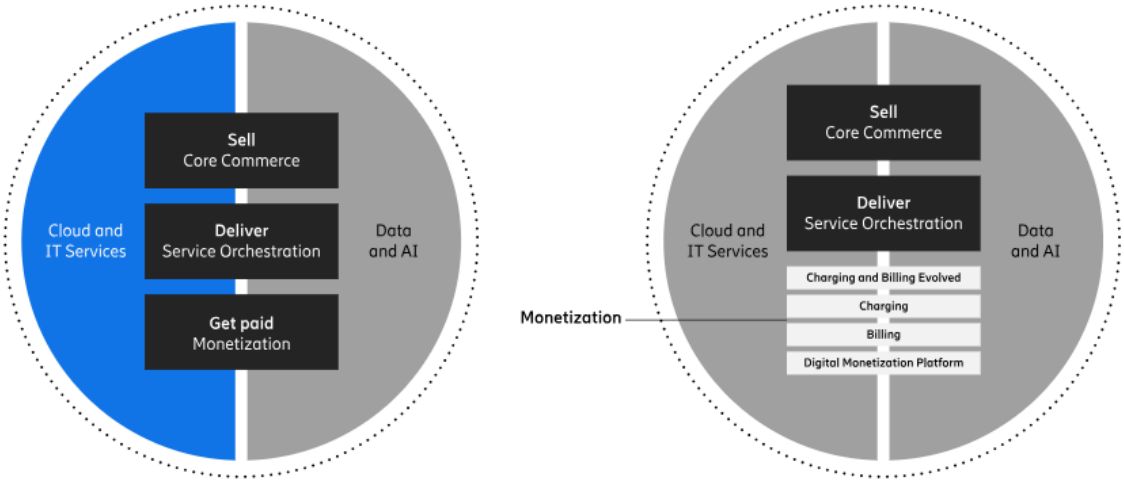


Figure4.1

With a basic understanding of the principles of GPON we can now embark on understanding the relationship between GPON and the system to be implemented , and to do that we need to understand the system's architecture.
The key concept of the figure4.1 above is the creation of a specialized yet wholly unified approach to building systems where in one hand the OSS (Operations Support System) is the backbone of an internet subscription operation that ensures smooth operations of the business [2] , i.e the concurrent nature of my system pipeline for all users from accessing the GUEST portal (to be discussed) to ensuring a streamlined user interface as users navigate across the application in the form the user interface to even ensuring a reliable connection to the implemented internet through protocols and system architecture. BSS (Business Support Service) is the infrastructure that allows all the OSS operations can flow, i.e the engine to my car so to speak, were the BSS is the former while the OSS is the later, it subsequently handles billing, charging, mediation, and order management – making sure you can sell, deliver, and get paid for your services [2].

Thus, paying homage to the architecture above I therefore made a topological system network to meet these demands for my system, with one half representing the Billing subsystem while the other act as the operational pipeline as figure 4.4[2]
And with those key principles I hence embarked into the understanding and implementation of a reliable network. A network is simply a group of interconnected devices communication with each other [1] the aforementioned devices could range from as basic as a pair of telephones linking neighbors' houses to a server each fulfilling the same purpose; providing access to information, the analogous nature of the system network is to show how data is generally communicated and transmitted across various system architecture , in my research I had learnt of two network architecture methods , *mesh and star network architectures.* Figure4.2 Represents my GPON billing system implemented as a star network where, BSS server (Server-PT) represents the embedded billing pipeline that handles payments, while the other half of the network represents the Operational pipeline for the system.

Taking this architecture into consideration, I arrived at a noticeable disadvantage of star network that severed or faulty paths will always lead to a lost connection in data communication.[1]
The lack of redundancy in star network, thus brings about unreliability if a route or channel is broken thus the need for a redundant mesh network as figure 4.3

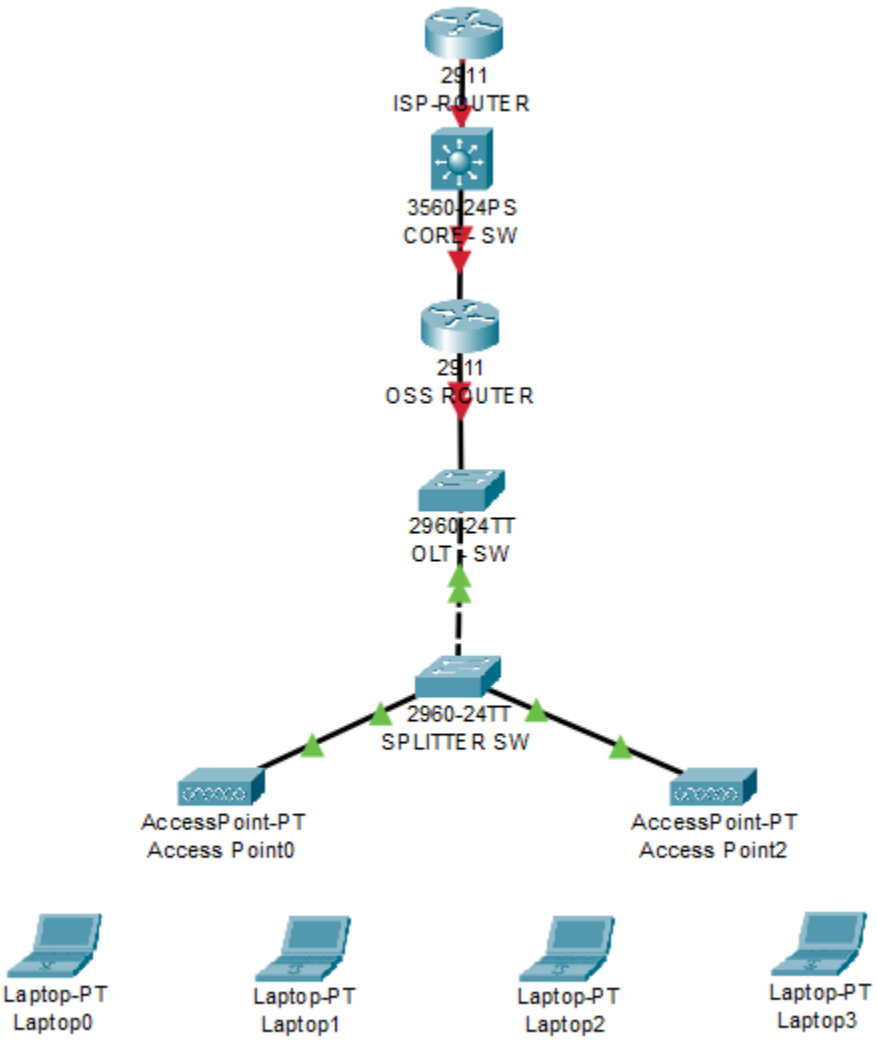


Figure 4.2: SIMPLE UNCONFIGURED STAR NETWORK IMPLEMENTATION

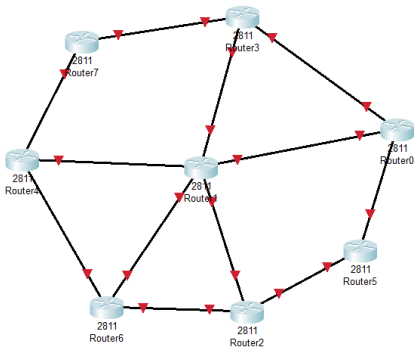


Figure 4.3 :SIMPLE UNCONFIGURED MESH NETWORK IMPLEMENTATION

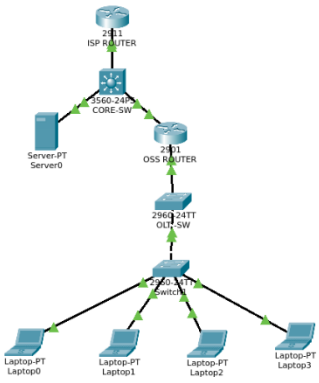


Figure 4.4 :A UNIFIED MESH-STAR NETWORK IMPLEMENTATION

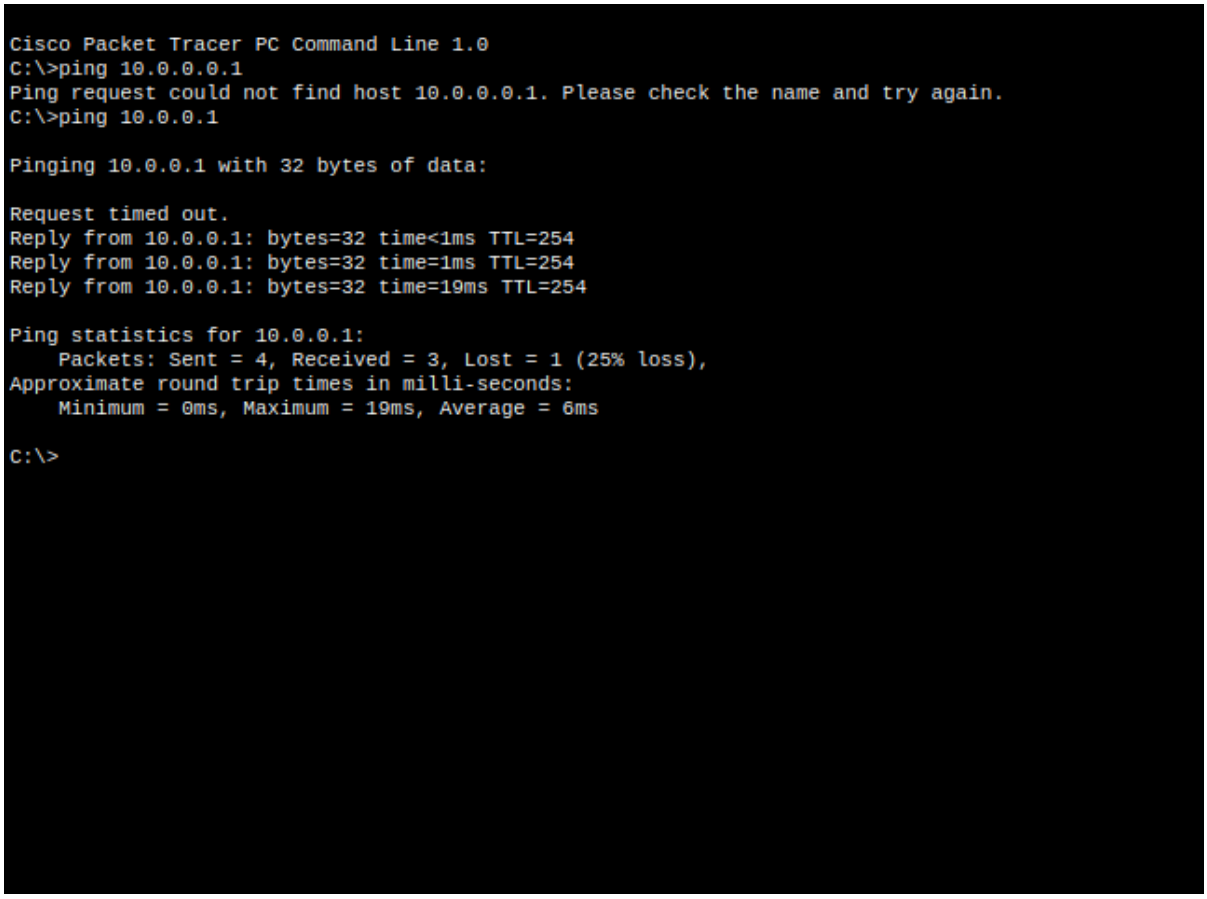


Figure 4.5 :PING TEST FOR MY NETWORK ARCHITECTURE

after conclusively creating that topological network architecture as a section of a Mesh ; Star Network I conducted a ping test to conclusively deduce that apart from the 25% (intentionally omitted packet loss from the Server-PT) a practical and implementable system was archived.

With the 75% packet Received (pinged from each laptop) represents the OSS pipeline from the ISP router to the consumer and 25% representing a ”virtual” BSS pipeline.

SECTION 5: GAP / PROBLEM SOLVED

The highly detailed approached to creating and implementing the project wholistically came from a realization that the pre-existing systems had a number of vulnerabilities such as poor infrastructure , poor security protocols , not scalable , not affordable and ultimately not practical to implement for the present market ; a need for a standalone system that overcomes all those faults while ensuring autonomy is a prominent cornerstone of the project and thus the problem statement was ultimately tackled from a technical and an economic lens .

SECTION 6: DESIGN APPROACH

Multiple design approaches were therefore researched so as to meet the aforementioned demands. Each approach unfortunately came about as a counter measure of a lack of implementation of the

ideal mode / method of implementation.

6.1 IDEAL METHOD OF IMPLEMENTATION

The original hardware specification for this project was built around three Ubiquiti products. The UAP-AC-M-PRO outdoor mesh access point was selected to provide WiFi coverage because its mesh capability meant it could extend coverage across a wider area without needing a wired connection back to the gateway at every installation point [3]. The Edge Router ER-6P was chosen as the network gateway because it offers gigabit routing performance alongside an SFP fiber port, which is necessary for connecting to a GPON uplink [4]. Bringing these two components together and managing them centrally required the UniFi Cloud Key Gen2 — a small dedicated controller device that runs the UniFi Network Application and provides the API through which the billing backend would authorize and revoke guest access [6]. The plan was straightforward in principle: the billing backend would communicate with the Cloud Key over the local network, the Cloud Key would push commands to the Edge Router and the access point, and the whole system would function as a unified, automated WiFi billing platform. That plan depended entirely on the Cloud Key being functional.

6.2 EMULATED OPERATIING SYSTEMS

The Cloud Key unit available for this project was not operational. Without it, there was no central controller, which meant the access points could not be configured, the guest portal policy could not be set, and the API calls from the billing backend had nowhere to go. This was not a trivial obstacle — the UniFi ecosystem is designed to be controller-dependent, and attempting to configure devices without one is not straightforward.

One early workaround considered was running the UniFi Network Application inside a virtual machine on a Windows laptop. Oracle VirtualBox was set up with an Ubuntu guest[6], and the UniFi software was installed within it. The system ran, but it was immediately clear that this arrangement had a fundamental problem: the virtual machine was completely dependent on the host laptop being switched on and running. Any time the laptop went to sleep, the network controller went offline, and with it the ability to manage the access points or process authorization commands from the billing backend. A billing system that stops working whenever the developer's laptop lid closes is not a credible prototype, so this approach was abandoned.

6.3 DESIGNING AROUND THE CLOUD KEY

Four alternative approaches were considered once the virtualization option was ruled out. The first, and ultimately the one adopted, was deploying the UniFi Network Application directly on a Linux server — either a physical machine or a cloud-hosted virtual private server. This provides a persistent, always-on controller environment that is not dependent on a personal device. The second option was containerizing the controller using Docker, which would make deployment and recovery faster and more reproducible. Docker was evaluated and found to be a viable approach, but since the server environment was already being configured for the Flask application and PostgreSQL, adding Docker introduced additional complexity without a compelling advantage at the prototype scale [7].

The third option was running the controller on a Raspberry Pi. This is a popular approach in hobbyist networking communities because the Pi is cheap, consumes very little power, and can run

continuously without the thermal and noise concerns of a full server. The limitation is throughput: a Raspberry Pi 4 has sufficient processing power for a small number of concurrent clients, but it would become a bottleneck in any deployment with significant concurrent traffic. For a prototype this is acceptable, but it would not be the right choice for a production system intended to serve a large number of simultaneous users.

The fourth option was using a cloud-hosted VPS to run the controller, which would add geographic flexibility — the controller could be accessed and managed from anywhere — but at the cost of a recurring subscription fee and the latency introduced by routing management traffic through a remote server rather than handling it locally.

6.4 ALTERNATIVE WORKAROUNDS

The software-based controller on a dedicated Linux server was selected as the best balance of cost, performance, and reliability for this project. Running the UniFi Network Application on the same Ubuntu Server machine that hosts the Flask backend eliminated an entire category of networking complexity: all communication between the billing system and the controller happens over the loopback interface, which is fast, reliable, and does not require any firewall rules to permit

Beyond solving the immediate hardware problem, this approach actually produced a better architecture than the original Cloud Key plan would have. With both systems on the same machine, the integration between the billing backend and the network controller is tighter and more efficient. There is no network hop for API calls, no certificate management issue across separate devices, and no risk that a network problem between the billing server and the controller could leave a paying subscriber without access. The constraint that arose from the Cloud Key failure turned out to be a useful forcing function that produced a cleaner and more integrated design.

6.5 THE IMPLEMENTED DESIGN

For the implementation of this project, a software-based controller hosted on a Linux system was selected as the most suitable alternative. This decision was guided by the need for:

1. Cost efficiency (no additional hardware required)
2. High performance and reliability
3. Ease of integration with the backend system
4. Full access to UniFi management capabilities

The chosen solution successfully replicated the core functionalities of the Cloud Key, enabling seamless configuration, monitoring, and management of UniFi devices within the network.

Designing around the absence of the Cloud Key introduced a more flexible and scalable architecture. Unlike the fixed hardware limitations of the Cloud Key, the software-based solution allowed for:

1. Integration with the backend billing system
2. Enhanced control over system resources
3. Improved adaptability for future upgrades

This modification aligns with the broader objective of the project, which is to develop a modular and automated network management system that integrates both billing and network control functionalities.

SECTION7: METHODOLOGY

This section documents the systematic procedures, tools, and technical decisions employed in constructing the automated GPON WiFi billing system. The integration spans UniFi wireless infrastructure, PayHero-mediated M-Pesa STK Push payments, a Flask-based Python backend, and PostgreSQL persistent storage, all deployed on Ubuntu Server 22.04 LTS

7.1 SYSTEM ENVIRONMENT AND DEPLOYMENT PLATFORM

Ubuntu Server 22.04 LTS was the operating system used throughout. The decision to use this particular version came down to three practical factors: it is free and open source, it has a five-year official support window which means security patches will keep coming without requiring a major OS migration, and it is the platform that most real-world Linux server tutorials and package repositories target [8]. Working on Ubuntu meant that almost every problem encountered during setup already had a documented solution online, which saved a meaningful amount of time.

Assigning the server a static IP address was one of the first configuration steps and also one of the most important. The UniFi access points need to know where to send users when they first connect to the guest network — they do this by redirecting HTTP traffic to a URL configured in the controller, and that URL has to point to a fixed address. If the server's address changed with each DHCP lease renewal, the redirect would break and users would land on an error page rather than the payment portal. Ubuntu uses a tool called Netplan to manage network interfaces. The relevant configuration file lives in the /etc/netplan/ directory, and editing it to assign a fixed IPv4 address, gateway, and DNS server — then running netplan apply to make the change take effect — is a one-time operation that the server then respects across reboots.

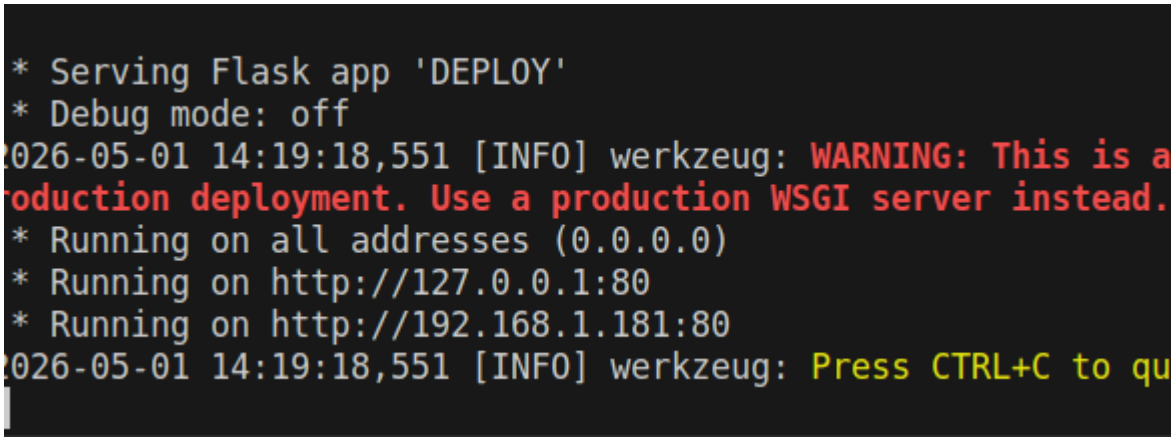


Figure 7.1 SERVER STATIC IP

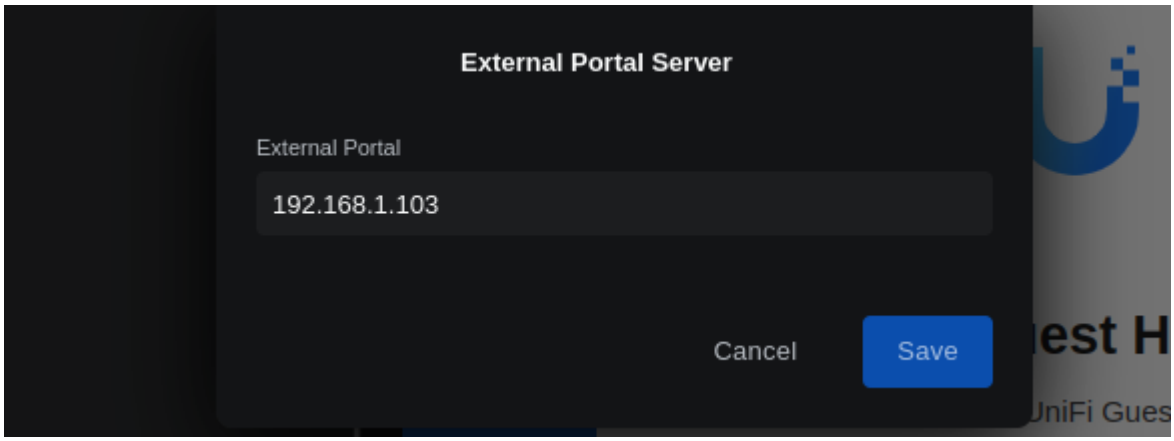


Figure 7.2 SERVER STATIC IP ASSIGNED TO UNIFI NETWORK APPLICATION

7.2 SOFTWARE INSTALLATION AND DEPEDENCY MANAGEMENT

7.2.1 Python and the Virtual Environment

Python 3.10 is included in Ubuntu 22.04 by default[9], but pip — the package manager needed to install additional libraries — is not. Installing it via APT takes one command. Once pip was available, a virtual environment was created inside the project directory. The purpose of a virtual environment is to keep all the packages installed for this project contained within a single folder, separate from anything installed at the system level. This matters because it means the project's dependencies can be updated or changed without affecting any other Python software on the machine, and the exact set of packages with their versions can be exported to a requirements.txt file, making it possible to recreate the environment exactly on a different machine. Every subsequent pip install command was run with the virtual environment active, so all packages landed in the right place.

7.2.2 Flask and the Supporting Libraries

Flask is a Python web framework that was installed via pip into the virtual environment. It is the component that handles all the incoming HTTP requests — from the subscriber's browser loading the login page, to the PayHero server sending the payment callback, to the polling requests that check transaction status[10]. Flask pulls in several supporting libraries automatically: Werkzeug, which does the actual work of parsing HTTP requests; Jinja2, which renders HTML pages by filling templates with data from the database; and Click, which handles the command-line options. Beyond Flask itself, three other packages were installed. Psycopg2-binary is the Python driver for PostgreSQL[11] — without it, the Flask application cannot talk to the database at all. The requests library is used every time the backend needs to make an outbound HTTP call, specifically when triggering the STK push through PayHero and when sending authorization commands to the UniFi controller. Python-dotenv loads the configuration values from a .env file so that things like the database password and API keys are not written directly into the application code.

7.2.3 PostgreSQL Setup and Database Schema

PostgreSQL 14 was installed from Ubuntu's repositories. After installation, it starts automatically and creates a default system user called postgres. To set up the project database, the postgres user was used to open the psql terminal, create a database called wifi_portal, and create a dedicated application user called portal_user with a password and full rights over that database. The reason for creating a separate user rather than using the postgres superuser for everything is that if the Flask application were somehow compromised, the attacker's access to the database would be limited to whatever portal_user can do — which is just reading and writing data in wifi_portal, not modifying the PostgreSQL system itself. The database schema consists of four tables. The users table holds subscriber account records — an ID, an email address, and a hashed version of the password so plaintext credentials are never stored anywhere. The plans table stores the WiFi packages on offer, with columns for the name, price in Kenyan Shillings, duration in minutes, bandwidth limit, and a flag indicating whether the plan is currently available. The transactions table is the most active table in operation — it records every payment attempt from the moment the user selects a plan through to the final success or failure confirmation from PayHero. Each row links a user, a plan, a phone number, a unique reference UUID, and a status value. The sessions table tracks which devices currently have active internet access, recording the MAC address, the time the session started, the time it is due to expire, and a flag that the background scheduler uses to know when revocation is needed.

```
mumo@mumo-HP-EliteBook-840-G3:~/DEPLOY$ psql postgresql://postgres:postgres@localhost:5432/wifi_portal
psql (16.0 (Ubuntu 16.0-1.pgdg22.04+1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

wifi_portal=# \dt
              List of relations
 Schema |      Name       | Type  |  Owner
--------+-----------------+-------+----------
 public | plans           | table | postgres
 public | portal_sessions | table | postgres
 public | transactions    | table | postgres
 public | users           | table | postgres
(4 rows)
```

Figure 7.3 POSTGRES RELATIONAL TABLE

7.2.4 Running the Flask Server

During development the Flask application was started directly from the command line using the Python interpreter, with the virtual environment active. The application was configured to bind to 0.0.0.0 on port 5000 or port 80 (in my setup), which means it listens on all the server's network interfaces rather than just the local loopback, making it reachable from any device on the same network. Debug mode was turned off for all testing that involved real devices or real payment transactions, because leaving it on exposes the interactive debugger to anyone who triggers an unhandled error, which is a security risk. For a real deployment beyond the prototype, Flask's built-in server would be replaced with Gunicorn[12], a proper application server that can handle multiple concurrent requests across a pool of worker processes. Nginx would sit in front of Gunicorn to handle HTTPS connections and serve any static files. A systemd service unit would ensure both processes restart automatically if the machine is rebooted. That full production stack was not the focus of this project, but the architecture was designed with it in mind so the transition would not

require rebuilding from scratch.

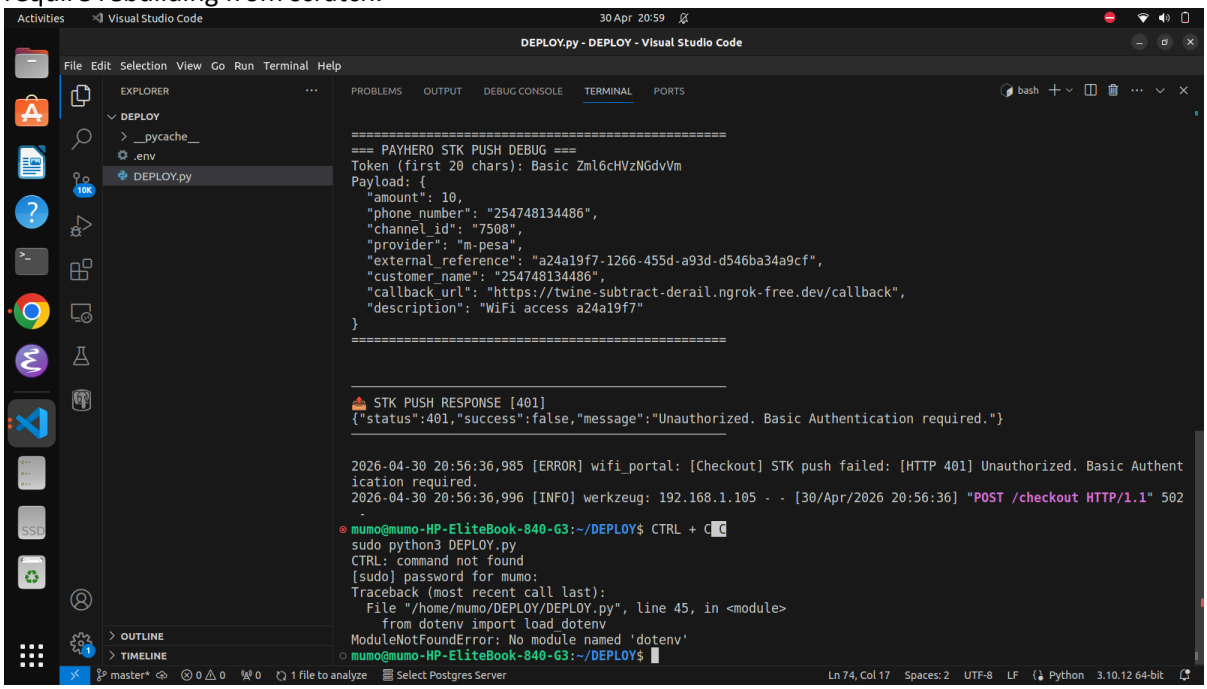


Figure 7.4 FLASK INITIALIZATION IN LINUX

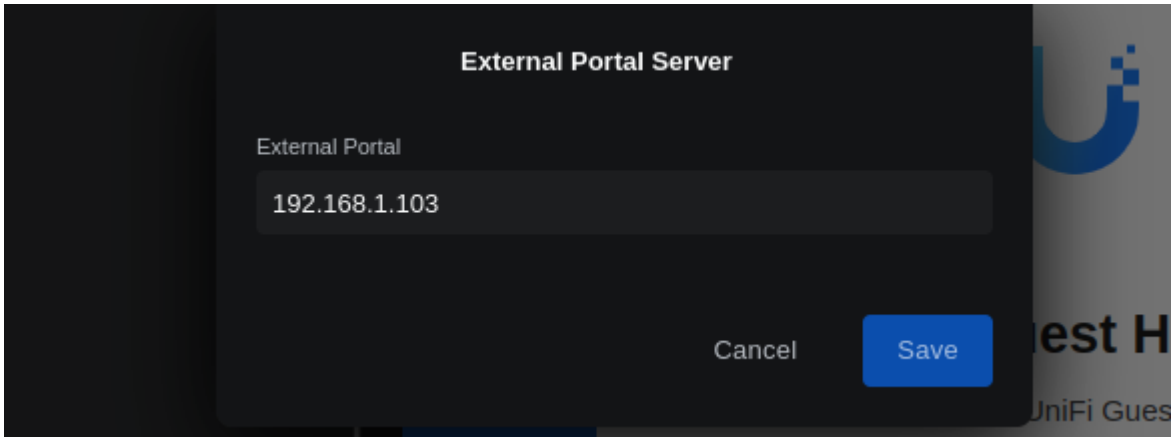
7.3 End-to-End System Workflow

The operational workflow of the system spans multiple architectural layers and progresses through a well-defined sequence of events; from the moment a subscriber connects their device to the WiFi network through to the point at which internet access is granted following successful payment. This section documents each step in that sequence with technical precision, identifying the responsible system component, the protocol or mechanism used, and the data exchanged at each transition.

7.3.1 Network Connection and Captive Portal Redirection

When a subscriber connects their phone or laptop to the guest WiFi network, the UniFi Access Point associates the device with the guest VLAN and the DHCP server assigns it an IP address. At this point the device has a local network address but no internet access — the UniFi gateway blocks all outbound traffic for unauthenticated devices. Most mobile operating systems try to confirm internet connectivity automatically when connecting to WiFi, so within a second or two of joining the network, the device is already generating an HTTP request. That request hits the UniFi gateway, which intercepts it and sends back an HTTP 302 redirect pointing at the captive portal running on the Flask server. The redirect URL carries two pieces of information as query parameters: the

device's MAC address and the URL the device was originally trying to reach. The Flask login route reads the MAC address out of that URL and stores it in the session. Every subsequent step in the payment flow depends on that MAC address being available, because it is what the UniFi controller uses to identify which device should receive internet access once payment is confirmed



AFOREMENTIONED FIGURE 7.1

7.3.2 User Registration and Authentication

The captive portal landing page gives the user two options: sign in with an existing account or register a new one. Both paths POST a form to the Flask backend. For new registrations, the backend checks that the email address is valid, confirms it does not already exist in the users table, hashes the password, and creates the record. For sign-ins, the submitted password is hashed and compared against the stored value. The comparison function takes the same amount of time regardless of whether the email exists in the database, which closes a timing side-channel that could otherwise allow an attacker to enumerate registered email addresses. Once the user is authenticated, a session cookie is set in their browser, and from that point the server can recognize them across page loads without requiring them to log in again. The MAC address captured at the redirect step is also saved into the session at this point, linking the authenticated user identity to the specific device they

are browsing from.

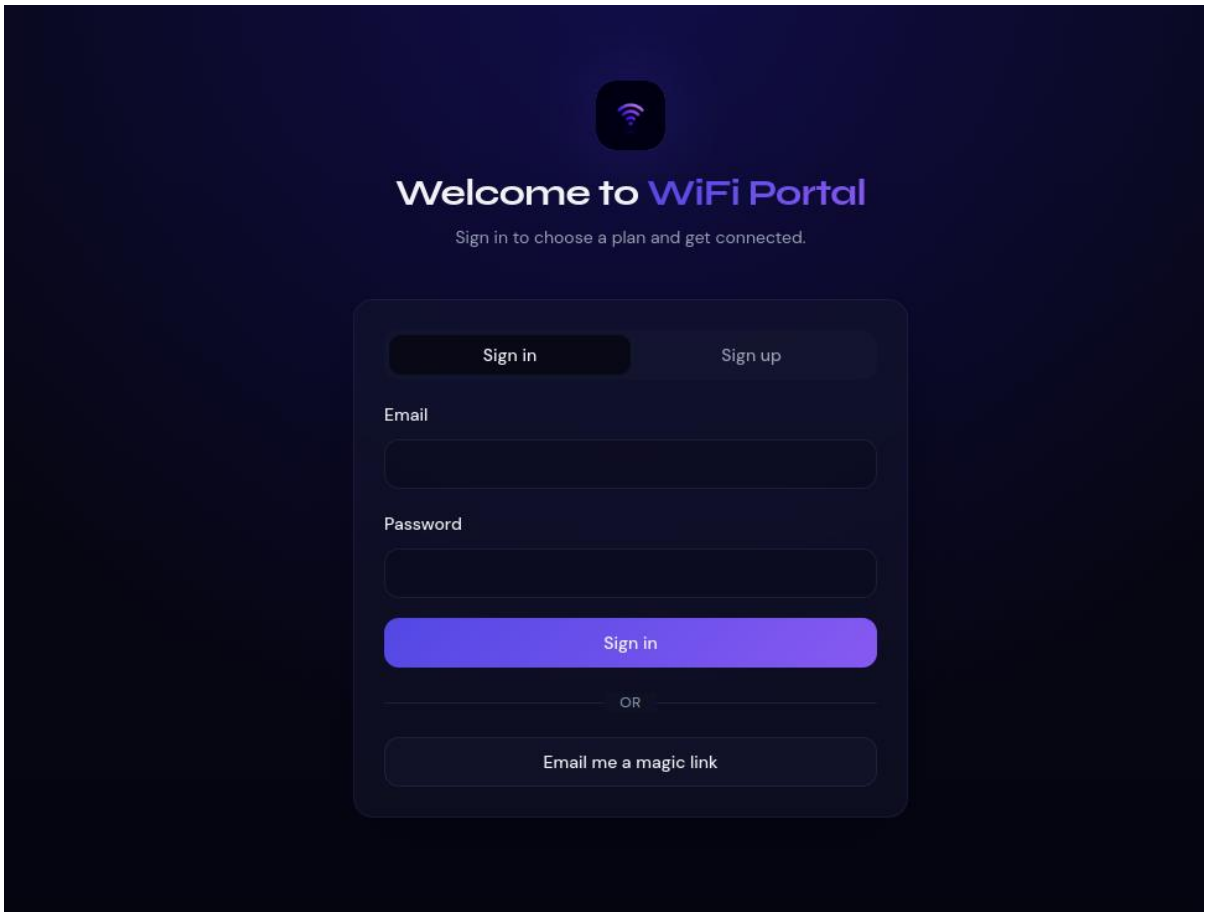


Figure 7.5 Guest Portal Landing Page

7.3.3 Plan Selection

After logging in, the user is shown the plan selection page. The Flask handler queries the plans table, retrieves all rows where the active flag is true, and passes them to a Jinja2 template that renders each one as a card showing the name, duration, speed, and price. When the user clicks to select a plan, the browser POSTs the plan's ID back to the server. The handler retrieves the full plan record, stores the price and duration in the session, and creates a new row in the transactions table with a status of pending. The UUID generated for this transaction — using Python's uuid module — is the external reference that will later be used to match the PayHero payment callback back to this specific purchase. With the transaction created, the user is redirected to the checkout page to

enter their phone number

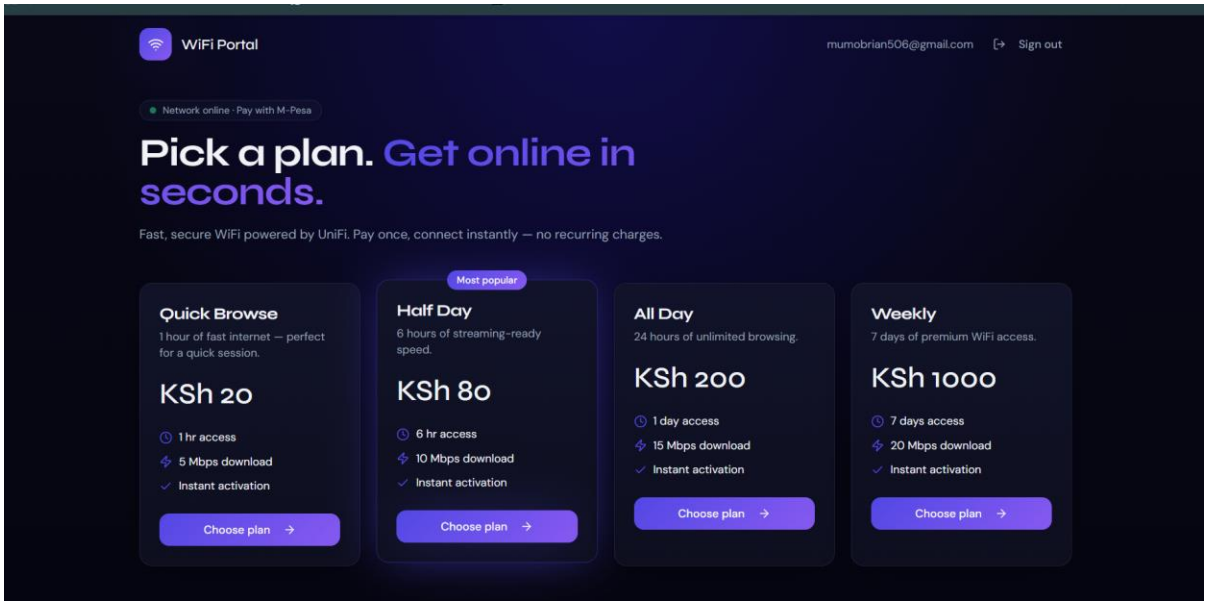


Figure 7.6 Plan Selection Page

7.3.4. STK Push Payment Initiation

The checkout page asks the user for the M-Pesa number they want to pay from. Before doing anything with it, the backend normalizes the format: numbers entered with a leading zero, such as 0712345678, are converted to the international format 254712345678 by substituting the zero with the Kenya country code 254. A regular expression then confirms the result is a valid 12-digit Kenyan mobile number before the request proceeds. If the number fails validation, the user is shown an error message immediately without any API call being made.

Once the number is confirmed as valid, the backend builds a JSON request body containing the payment amount from the plan, the normalized phone number, the PayHero channel ID identifying this merchant, the external reference UUID from the transaction record, and the callback URL — a publicly accessible HTTPS address hosted on this server where PayHero will send the result. This request is sent to the PayHero API with an authorization header carrying the API key. PayHero passes the request on to Safaricom's M-Pesa infrastructure, which sends an STK push prompt directly to the user's phone. The user sees a payment confirmation screen asking for their PIN, they enter it, and M-Pesa processes the transaction. While this is happening, the browser has been redirected to a waiting page that checks back with the server every few seconds to see whether the payment has resolved

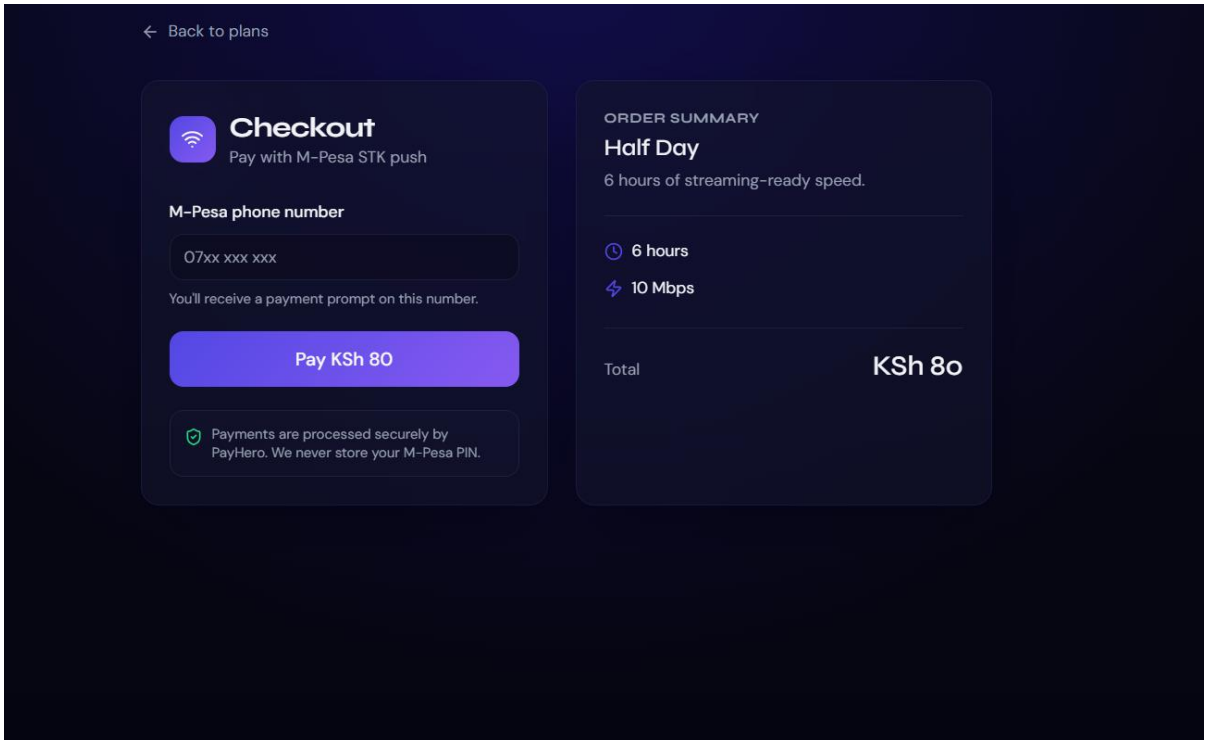


Figure 7.7 Checkout Page

7.3.5 Payment Callback Handling and Database Update

PayHero sends the payment result to the callback URL as an HTTP POST with a JSON body. The Flask callback handler receives this, logs the full payload for audit purposes, and extracts two key pieces of information: the external reference UUID that identifies the transaction, and the payment status. PayHero can send callbacks in slightly different formats depending on whether the M-Pesa response is nested or flat, so the handler checks both the top-level status field and the nested response object to find the status value. A result indicating success is mapped to the string 'success'; anything indicating failure maps to 'failed'.

Before updating the database, the handler retrieves the existing transaction record and checks whether it has already been processed. PayHero sometimes re-sends a callback if it does not receive a timely acknowledgement, so this check prevents the same payment from triggering duplicate authorisations. If the record is still in the pending state, the handler updates it to the new status. The handler then returns HTTP 200 to acknowledge receipt before doing anything else — returning any other status code would cause PayHero to retry the callback repeatedly.

```
📥 PAYHERO CALLBACK: {'status': True, 'response': {'MerchantRequestID': 'b424-4bec-973f-fac88b9b7d19
27030399', 'CheckoutRequestID': 'ws_CO_030520261627462857481344 86', 'ResultCode': 0, 'Amount': 30, '
MpesaReceiptNumber': 'UE3DT2SFDP', 'Phone': '254748134486', 'ExternalReference': '10b0b58a-ff78-4516
-9a7e-3c94ffd3b9ef', 'Status': 'Success', 'ResultDesc': 'The service request is processed successful
ly.', 'ServiceWalletBalance': 0, 'PaymentWalletBalance': 0, 'ChannelID': 7508}, 'forward_url': ''}
2026-05-03 16:27:59,060 [INFO] werkzeug: 192.168.1.102 - - [03/May/2026 16:27:59] "GET /status_api/1
0b0b58a-ff78-4516-9a7e-3c94ffd3b9ef HTTP/1.1" 200 -
2026-05-03 16:28:00,287 [INFO] werkzeug: 127.0.0.1 - - [03/May/2026 16:28:00] "POST /callback HTTP/1
.1" 200 -
2026-05-03 16:28:01,324 [INFO] werkzeug: 192.168.1.102 - - [03/May/2026 16:28:01] "GET /status_api/1
0b0b58a-ff78-4516-9a7e-3c94ffd3b9ef HTTP/1.1" 200 -
2026-05-03 16:28:03,715 [INFO] werkzeug: 192.168.1.102 - - [03/May/2026 16:28:03] "GET /success HTTP
/1.1" 200 -
```

```
ngrok                                                                    (Ctrl+C to quit)

Take our ngrok in production survey! https://forms.gle/aXiBFWzEA36DudFn6

Session Status                online
Account                       mumobrian506@gmail.com (Plan: Free)
Version                       3.39.1
Region                        India (in)
Latency                       872ms
Web Interface                 http://127.0.0.1:4040
Forwarding                    https://twine-subtract-derail.ngrok-free.dev -> http://localhost:80

Connections                   ttl     opn     rt1     rt5     p50     p90
                              3       0       0.00    0.00    0.07    1.34

HTTP Requests
-------------

16:27:58.962 EAT POST /callback                 200 OK
16:27:26.914 EAT POST /callback                 200 OK
16:27:19.611 EAT POST /callback                 200 OK
```

Figure 7.8 & 7.9Post Callback Clarification for LOCALHOST & NGROK

7.3.6 UniFi Authorization and Internet Access Grant

When the payment status is updated to success, the handler immediately makes two API calls to the UniFi controller. The first is an authentication call to establish an administrative session. The second sends the authorization command, passing the device MAC address from the session and the plan duration from the transaction record. The UniFi controller grants internet access for that device, and the Flask backend creates a record in the sessions table with the start time and the computed expiry time. Back in the subscriber's browser, the polling loop that has been running every three seconds detects the success status on its next check and redirects the browser to a confirmation page. The user can now browse freely.

7.4 Automation Architecture

7.4.1 Event-Driven Callback Architecture

The key architectural decision that makes the system fast is using PayHero's webhook callback rather than polling the PayHero API to check for payment results. In a polling model, the server would need to send a query to PayHero every few seconds and wait for the response. Each query takes time, and the user waits for the next polling cycle before anything happens. With a webhook, PayHero pushes the result to the server the moment the payment is settled, and the server acts on it immediately. In practice this means the time between the user confirming their PIN and the server receiving the callback is typically two to five seconds, almost entirely determined by how long M-Pesa takes to process the transaction rather than any delay introduced by the architecture.

The callback handler is designed to be stateless — it reads what it needs from the database, does its work, writes the results back, and returns. Nothing important is held in memory between requests. This means that if the server were to restart at an inconvenient moment, no payment data would be lost, and reprocessing from the database state would be possible.

7.4.2 Polling Fallback for the Subscriber Interface

While the server-side confirmation is event-driven, the user's browser cannot receive unsolicited updates without a persistent connection. Establishing a WebSocket connection or using Server-Sent Events would require the browser to maintain an open connection throughout the payment wait, which is fragile on mobile connections and not universally supported on low-end devices. Instead, a simple JavaScript function on the waiting page sends a lightweight GET request to the server every three seconds asking for the current transaction status. When the status comes back as success or failed, the JavaScript stops polling and either redirects the browser or shows an error. If a polling request fails because the connection drops briefly, the function just waits for the next interval and tries again. This makes the approach robust without being complicated.

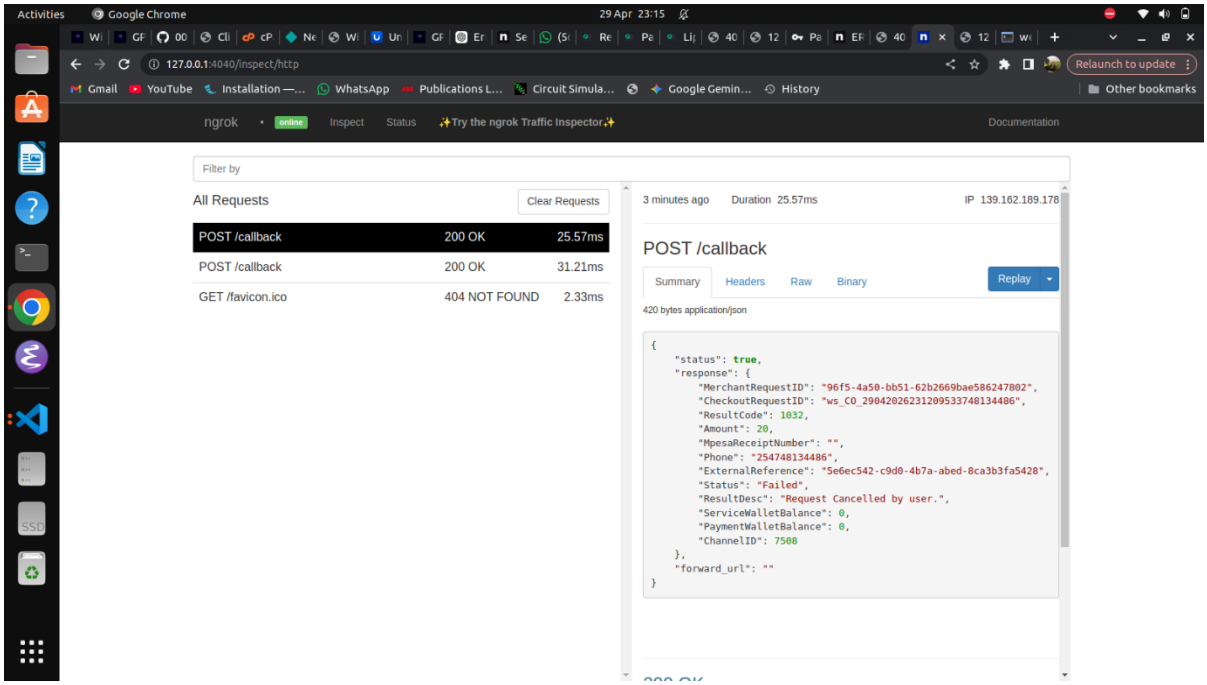


Figure 7.10 Post Callback Clarification for NGROK

7.4.3 Session Expiry and Automated Revocation

Access is not just granted — it is also revoked automatically when it expires. The APScheduler library runs a background task inside the Flask application on a one-minute cycle. Each time it runs, it queries the sessions table for rows where the expiry timestamp has passed and the session is still marked active. For each such row, it calls the UniFi controller API to unauthorize the device, then updates the session record to mark it as inactive. Without this mechanism, a user who paid for one hour of access and then left without explicitly logging out would continue to have internet access indefinitely, which is both a revenue loss and a fairness issue for other users sharing the same bandwidth.

7.5 UniFi Controller Integration

7.5.1 UniFi Controller Architecture and Role

The UniFi Controller is the management layer for all the Ubiquiti network hardware. In the context of this project, it serves two roles. First, it is responsible for the guest network policy: it defines which VLAN guest devices are placed into, what the captive portal redirect URL is, and what firewall rules the gateway applies to block internet traffic for unauthenticated devices. Second, it exposes a REST API that the Flask backend calls whenever it needs to grant or revoke access for a specific device. This means the billing backend never has to touch the gateway's firewall rules directly — it just tells the controller what to do, and the controller handles the implementation details.

For this project, the controller runs on the same Ubuntu server as the Flask application. That co-location means all API calls between the two travels over the loopback interface, which is both fast and avoids any requirement to open additional firewall ports for controller-to-billing communication.

7.5.2 API Authentication

Every sequence of API calls to the UniFi controller begins with a login request. The Flask access function sends an HTTP POST to the controller's /api/login endpoint with a JSON body containing the username and password of a dedicated API account created specifically for this integration. Using a dedicated account rather than the main administrator account limits the potential damage if those credentials were ever exposed: the API account only has the permissions needed to manage guest authorizations, nothing more. The controller responds with a session cookie that must be included in all subsequent requests. The Flask code uses a requests.Session object to store and automatically send that cookie, so it does not need to be tracked manually. If the session expires and a subsequent call returns HTTP 401, the code re-authenticates and retries the original request transparently.

The controller uses a self-signed TLS certificate by default, which causes Python's requests library to raise a verification error. In a trusted private network environment, suppressing that verification is an acceptable practical decision. A production deployment would use a properly signed certificate to avoid this workaround.

7.5.3 MAC-Based Guest Authorization

Guest device authorization is done by posting a JSON command to the controller's site management endpoint at /api/s/default/cmd/stamgr. The command body specifies the operation — authorize-guest to grant access or unauthorize-guest to revoke it — along with the device MAC address and, for authorization, the number of minutes of access being granted. The controller translates this into the appropriate change in the gateway's iptables rules: an authorized device's traffic is permitted to pass through to the internet, while an unauthorized device's traffic continues to be intercepted and redirected to the captive portal. This change takes effect within a second or two of the API call, which is why the user's browser, on its next polling check after successful payment, finds that internet access has already been granted.

7.5.4 Session Expiry and Authorization

Session expiry is enforced through the complementary unauthorize-guest command, which is issued by the APScheduler background task described in Section 3.4.3. The structure of this API call is analogous to the authorization call: an HTTP POST to the same /api/s/[site_name]/cmd/stamgr endpoint, with the command field set to unauthorize-guest and the mac field set to the MAC address of the device whose session has expired. The UniFi Controller processes this command by removing the guest authorization table entry and reinstating the iptables REDIRECT rule for the device's traffic, restoring the captive portal intercept state. If the subscriber attempts to access any internet resource after their session expires, their browser is once again redirected to the captive portal, where they are invited to purchase an additional plan. This behavior is fully automated, requiring no operator intervention, and ensures that subscribers do not receive internet access beyond the duration they have paid for.

7.6 Justification for the Selection of Python and Flask

7.6.1 Python as the Implementation Language

Python was chosen not because it is fashionable, but because the specific combination of tasks this project requires — database queries, HTTP API calls, cryptographic password hashing, background scheduling, and web request handling — all have mature, well-maintained Python libraries that work well together. Psycopg2 for the database, requests for HTTP calls, hashlib for password hashing, APScheduler for the background task, and Flask itself for the web layer form a coherent stack with no gaps that would require writing anything from scratch. Python's syntax is also genuinely readable, which matters in a final year project context where someone who did not write the code needs to be able to understand it from reading it.

Python's ubiquity on Ubuntu means it was already present on the server with no additional installation steps. The pip package manager and virtual environment tooling made dependency management straightforward. The available code quality tools — pylint for analysis, pytest for testing — meant the code could be checked systematically rather than relying entirely on manual review.

7.6.2 Flask as the Web Framework

Flask was chosen over Django after looking honestly at what the project actually needed from a web framework. Django is a comprehensive, opinionated framework that includes an ORM, an automatically generated admin interface, a forms library, and a rigid project structure. For a project with four database tables, no need for an auto-generated admin panel, and a preference for writing SQL directly rather than through an abstraction layer, all of that machinery is overhead that adds complexity without adding value. Flask provides the things that are actually needed here: URL routing, request context, session management, and Jinja2 templating. Everything else can be added as a specific library only if it is needed.

Flask's smaller footprint also makes the application's logic easier to follow from end to end. When debugging an issue with the payment callback, for example, it is straightforward to trace the request from the route definition through the handler function to the database call and back. Django's middleware stack and model layer would add several more layers to that trace.

7.6.3 Integration Capabilities and API Compatibility

Both external services used in this project — PayHero and the UniFi controller — communicate over HTTPS using JSON. Python's requests library handles both the JSON serialization of outgoing request bodies and the deserialization of incoming response bodies automatically, which means the application code can work directly with Python dictionaries rather than dealing with raw string formatting. The library also handles connection pooling, so repeated calls to the same endpoint reuse existing connections rather than establishing new ones each time. Psycopg2's parameterized query interface ensures that any data coming from user input — phone numbers, email addresses, plan IDs — is handled safely without any risk of SQL injection.

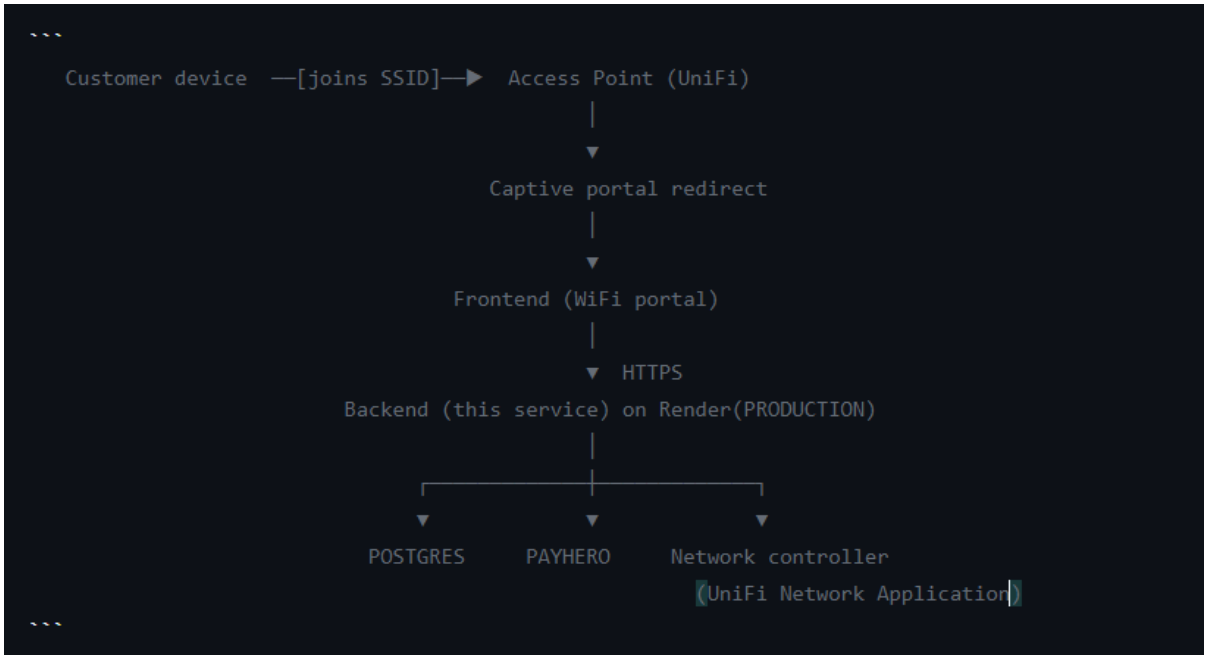


Figure 7.11 SYSTEM INTERGRATION FROM README FILE IN GITHUB

The complete system environment described in this chapter can be reproduced from a freshly installed Ubuntu Server 22.04 LTS image by following the installation sequence documented in Appendix A, which records every APT package installation command, pip install instruction, database initialization query, and environment configuration step in the exact order in which they were applied during the project implementation.

7.7: SUMMARY OF METHODOLOGY

The methodology described in this chapter constitutes a coherent and technically sound approach to the design and implementation of the automated WiFi captive portal system. The following table summarizes the key methodological decisions and their technical justifications.

Dimension	Decision	Justification
Operating System	Ubuntu Server 22.04 LTS	Long-term support, extensive package repositories, industry standard for server deployments
Backend Language	Python 3.10	Rich ecosystem, readable syntax, native Ubuntu support, mature API client libraries
Web Framework	Flask (micro-framework)	Minimal overhead, suitable for small route sets, clean integration with psycopg2 and requests

Database	PostgreSQL 14	ACID compliance, reliable for financial transaction records, parameterized query support
Payment Integration	PayHero STK Push (M-Pesa)	Dominant payment rail in Kenya, webhook callback model enables sub-5s provisioning
Network Enforcement	UniFi Controller REST API	MAC-level access control, programmatic authorization/unauthorization via JSON API
Automation Model	Event-driven callbacks + APScheduler	Eliminates polling latency on server side; scheduled task handles session expiry
Client Feedback	3-second HTTP polling from browser	Universal browser compatibility, no persistent connection required, graceful degradation
Security	UFW firewall, hashed passwords, parameterized SQL, dedicated API users	Defense-in-depth approach; principle of least privilege applied at every integration boundary

The methodology articulated in this chapter establishes a rigorous, reproducible, and technically justified foundation for the system implementation. The combination of Ubuntu Linux as the operating environment, Python and Flask as the application tier, PostgreSQL as the persistence layer, event-driven webhook processing as the automation mechanism, and the UniFi Controller REST API as the network enforcement interface collectively address the core engineering challenge: eliminating the manual handoff between payment confirmation and network access grant that characterizes conventional ISP billing operations. The chapters that follow present the detailed results of this implementation, evaluate its performance against the design objectives established in Chapter 1, and critically discuss the engineering trade-offs inherent in the selected approach.

SECTION 8: PROGRAMME DESIGN

This section explains how the software was structured internally — what the main building blocks are, what each one is responsible for, how they connect to the database, and why certain design choices were made at the code level. The system was built as a single Python file using Flask, which made the overall structure straightforward to navigate while keeping the codebase self-contained.

```
backend/
├── src/
│   ├── server.ts                 # Process entry: starts HTTP, handles signals
│   ├── app.ts                    # Express app factory: middleware + routes
│   │
│   ├── config/
│   │   ├── env.ts                # Zod-validated environment loader (fail-fast)
│   │   ├── logger.ts             # pino logger (pretty in dev, JSON in prod)
│   │   └── supabase.ts           # Supabase service-role client
│   │
│   ├── routes/
│   │   └── index.ts              # Wires URLs → middleware → controllers
│   │
│   ├── controllers/              # Thin HTTP layer: parses req, calls service, sends res
│   │   ├── payments.controller.ts
│   │   └── access.controller.ts
│   │
│   ├── services/                 # Business logic, talks to DB and Daraja
│   │   ├── PAYHERO.service.ts     # OAuth, STK push, callback parsing
│   │   ├── payments.service.ts   # Subscriptions + payments persistence
│   │   └── access.service.ts     # JWT issuance + verification
│   │
│   ├── middleware/
│   │   ├── auth.ts               # Verifies Supabase user JWT → req.userId
│   │   ├── rateLimit.ts          # Per-IP / per-phone rate limits
│   │   ├── validate.ts           # Generic Zod body/params validator
│   │   └── error.ts              # 404 + central error handler
│   │
│   └── utils/
│       ├── errors.ts             # HttpError + helpers (BadRequest, NotFound, …)
│       └── phone.ts              # Normalize KE phone numbers → 2547XXXXXXXX
```

Figure 8.1 BACKEND AND SRC(FRONTEND) FILE STRUCTURE INTERGRATION - PROTOTYPE

8.1 Why Python and Flask for This Application

Python was chosen because the tasks this application needs to perform — handling HTTP requests, querying a relational database, calling external REST APIs, hashing passwords, running a background scheduler — are all things for which well-maintained, widely-used Python libraries already exist. Writing those components from scratch would have taken significantly longer and introduced more opportunity for bugs. Flask was the natural framework choice because it imposes very little structure and gives the developer control over how the application is organized, which matters when the system needs to integrate with multiple external services in specific ways. A more prescriptive framework like Django would have required working around its conventions rather than with them.

One specific advantage of keeping everything in a single Flask file during the prototype phase is that the data flow through the system is easy to trace. When debugging the callback handler, for example, the code path from the incoming HTTP request through the database update to the UniFi API call is visible in a single file without navigating between modules. For a prototype that will be

demonstrated and examined, readability of this kind is genuinely useful.

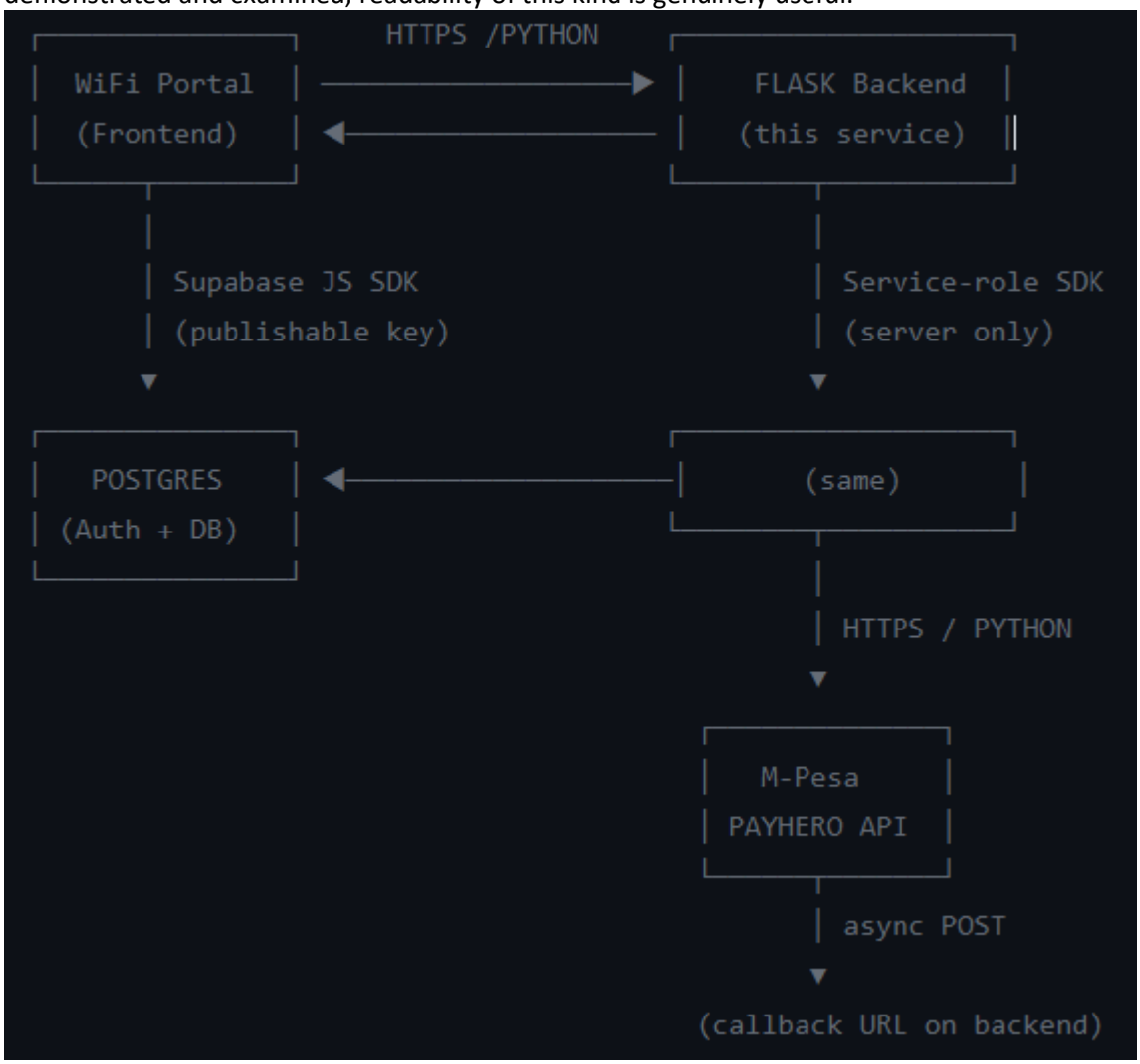


Figure 8.2 FLASK BACKEND INTERGRATION

8.2 JSON as the Communication Format

JSON was used as the data exchange format for all communication between the Flask backend and the two external services — PayHero and the UniFi controller. Both services accept JSON request bodies and return JSON responses, which Python's built-in json module handles without any additional dependencies. The requests library serialises Python dictionaries into JSON automatically when the json parameter is passed to a request call, and deserialises JSON response bodies into dictionaries on the way back in. Working with dictionaries directly rather than raw strings makes the code cleaner and less prone to formatting errors.

JSON was not used for data storage. The PostgreSQL database uses a relational schema with typed columns and foreign key constraints because that structure enforces data integrity in ways that a JSON document store cannot. The relationship between a user, a plan, and a transaction — with the constraint that a transaction must reference a valid user and a valid plan — is something the database enforces automatically at the schema level. Using JSON for storage would lose that guarantee. The two technologies serve different purposes: JSON for communication, relational tables for persistence.

8.3 The Main Functions and What They Do

The application is structured around a set of route handler functions and supporting utility functions. Each handler is responsible for one specific operation, and the utility functions are shared across handlers wherever the same logic is needed more than once.

The database query utility function is used by every part of the application that needs to read from or write to the database. It opens a connection, executes the provided SQL statement with the provided parameters, optionally fetches results, and commits the transaction if the operation is a write. Centralizing this logic means that every database interaction in the application follows the same pattern — the same error handling, the same connection management, the same commit behavior. Changing how connections are pooled, for example, is a one-place change rather than a change spread across every function that touches the database.

The user authentication functions handle registration and login. The registration handler checks for duplicate emails, hashes the submitted password using a secure one-way algorithm, and writes the new user record. The login handler retrieves the stored hash for the submitted email and compares it against a hash of the submitted password. These functions interact exclusively with the user's table.

The plan selection and checkout functions handle the creation of transaction records. When a user selects a plan, the checkout handler reads the plan details from the plans table, generates a UUID for the external reference, creates a pending transaction in the transactions table, and stores the relevant values in the session for use in subsequent steps. The phone number validation logic — converting from local to international format and checking the result against a regular expression — also lives in this handler.

The callback handler is the most consequential function in the system. It receives the payment result from PayHero, checks whether the transaction has already been processed, updates the status in the transactions table, and if the payment was successful, triggers the network authorization sequence. This function interacts with both the transactions table and the sessions table, and also makes outbound API calls to the UniFi controller.

The network authorization function handles the two-step process of logging into the UniFi controller API and sending the authorization command. It reads the device MAC address from the sessions table and the plan duration from the plans table to construct the authorization request. This function is also called by the background scheduler when revoking access at session expiry, with the command changed from authorize-guest to unauthorize-guest.

Administrative functions allow the operator to add new plans, remove existing ones, and view a summary of transaction revenue. These functions read from and write to the plans and transactions tables and are protected by a separate admin authentication check so that regular subscribers cannot access them.

8.4 How Functions Map to Database Tables

Each function has a clear and limited relationship with the database schema. Authentication functions work only with the user's table. Plan management functions — both the public-facing plan selection and the administrative plan creation and deletion functions — work with the plans table. The checkout, callback, and status-checking functions work primarily with the transactions table, with the callback function also writing to the sessions table when a payment succeeds. The UniFi

authorization function reads from the sessions table to get the MAC address and expiry time it needs to construct the API call.

This mapping ensures that each part of the system has a defined scope of responsibility. A bug in the callback handler, for example, cannot accidentally modify a user's account credentials because the callback handler has no code that touches the user's table. The clean separation between functions and tables makes the system easier to test, easier to debug, and easier to extend.

8.5 Database Table Structure

The users table is the starting point for everything. It stores a unique integer identifier, an email address with a uniqueness constraint so duplicate registrations are rejected at the database level, and the hashed password. A timestamp records when the account was created."),

The plans table defines the available WiFi packages. Each row has a name, a price in Kenyan Shillings as an integer, a duration in minutes, an optional bandwidth limit, and a boolean flag indicating whether the plan is currently on sale. Plans can be deactivated rather than deleted, which preserves the foreign key references from existing transactions.

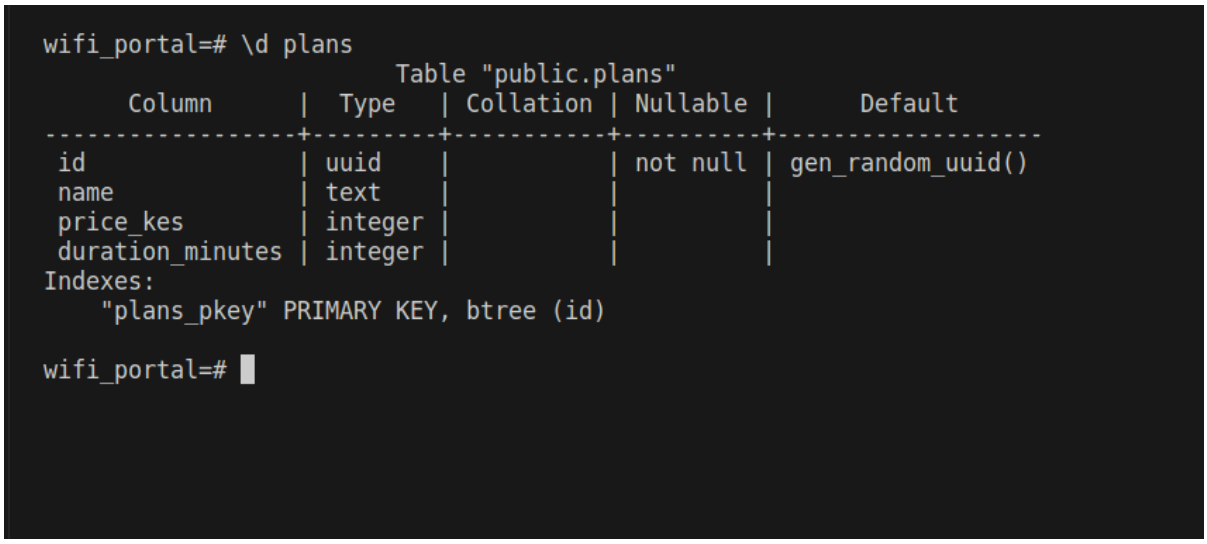


Figure 8.3 PLANS SQL TABLE

The transactions table is the most detailed. It stores the UUID external reference that ties the row to the PayHero payment [12], foreign keys referencing the user and plan, the M-Pesa phone number used for the payment, the amount charged, the current status as a string value, and timestamps for creation and last update. The external reference column has a unique constraint, so the database itself prevents duplicate transaction records for the same PayHero reference.

```
                                Table "public.transactions"
       Column       |           Type           | Collation | Nullable |      Defaul
--------------------+--------------------------+-----------+----------+------------
 id                 | uuid                     |           | not null | gen_random_
 external_reference | text                     |           |          |
 mac_address        | text                     |           |          |
 plan_id            | uuid                     |           |          |
 amount             | integer                  |           |          |
 status             | text                     |           |          |
 created_at         | timestamp with time zone |           |          | now()
 phone_number       | text                     |           |          |
 updated_at         | timestamp with time zone |           |          | now()
Indexes:
    "transactions_pkey" PRIMARY KEY, btree (id)
    "idx_tx_mac" btree (mac_address)
    "idx_tx_ref" btree (external_reference)
    "idx_tx_status" btree (status)
```

Figure 8.4 TRANSACTIONS SQL TABLE

The sessions table records active and historical access grants. Each row stores the device MAC address, a foreign key reference to the authorizing transaction, the session start time, the computed expiry time, and a boolean indicating whether the session is currently active. The background scheduler uses this table directly — it queries for active sessions with expired timestamps and updates the active flag to false after revoking access through the UniFi API.

```
wifi_portal=# \d portal_sessions
                          Table "public.portal_sessions"
     Column     |           Type           | Collation | Nullable |      Default
----------------+--------------------------+-----------+----------+-------------------
 id             | uuid                     |           | not null | gen_random_uuid()
 mac_address    | text                     |           | not null |
 transaction_id | uuid                     |           |          |
 plan_id        | uuid                     |           |          |
 expires_at     | timestamp with time zone |           |          |
 created_at     | timestamp with time zone |           |          | now()
Indexes:
    "portal_sessions_pkey" PRIMARY KEY, btree (id)
    "idx_sessions_mac" btree (mac_address)
    "portal_sessions_mac_address_key" UNIQUE CONSTRAINT, btree (mac_address)

wifi_portal=#
```

Figure 8.3 PORTAL-SESSIONS SQL TABLE

SECTION 9: BILL OF MATERIALS

	GPON BILLING SYSTEM
SUPPLY , TEST AND COMMISSION THE FOLLOWING :	

ITEM	DESCRIPTION	UNIT	QUANTITY	RATE	TOTAL
1	Ubiquiti UniFi AC Outdoor Mesh Pro Access Point (UAP-AC-M-PRO)	NO	1	39,000	39,000
2	Ubiquiti EdgeRouter ER-6P Gigabit Router with SFP	NO	1	51,000	51,000
3	Ubiquiti UniFi CloudKey Plus SSD (UCK-G2-SSD)	**NO**	**1**	**52,000**	**52,000**
4	**CAT6 CABLE AS 23 AWG (solid copper)4 Unshielded Twisted Pairs (UTP) or Shielded (FTP/SFTP) with a longitudinal separator/spline.**	**LM**	**305**	**LOT**	**25,000**

5	**Data router as Tenda F3 N300 300Mbps Wireless Router or approved alternative**	**No**	**1**	**1,500**	**7,500**
6	**Data Socket for RJ45** **As SKU TR5443** **Or CRABTREE OR APPROVED ALTERNATIVES**	**NO**	**5**	**1200**	**6,000**
TOTAL AMOUNT CARRIED FORWARD Ksh					**141,500**

SECTION 10: IMPLEMENTATION

10.1 Setting Up the Environment

The first practical task was getting the Ubuntu server into a state where the application could run. This involved installing Python's pip package manager, creating a virtual environment, installing PostgreSQL, and configuring the database with the schema described in Section 8. The application's configuration values — the database connection string, the PayHero API key[12], the UniFi controller address and credentials — were stored in a .env file that the application reads at startup using python-dotenv. This approach keeps credentials out of the application code and out of version control.

The Flask application was run on port 5000 during all development and testing. Ngrok was used to create a temporary public HTTPS URL tunneled to the local server, which was necessary because PayHero requires the callback URL to be publicly accessible over HTTPS. Safaricom's M-Pesa infrastructure will not send callback POST requests to a private IP address or a plain HTTP endpoint. Each test session required updating the callback URL in the PayHero configuration to match the current Ngrok URL, which was a small but recurring overhead during testing.

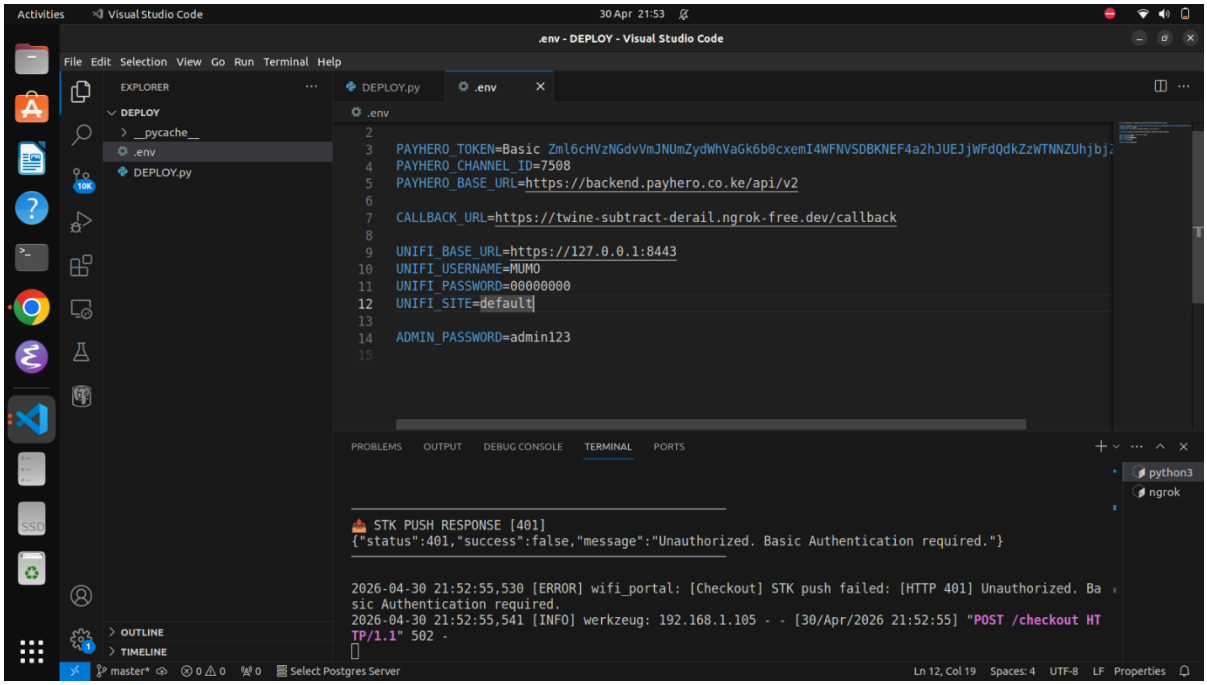


Figure 10.1 LOCAL ENVIRONMENT SETUP

10.2 The Flask Application Structure

The entire backend was implemented as a single Flask file. That choice was deliberate for the prototype stage: keeping everything in one place made it much faster to develop and debug, and made it easy to demonstrate the complete data flow to anyone reviewing the project. The file contains route definitions for every URL the application handles, the database query utility function, the user authentication logic, the plan selection and checkout handlers, the PayHero callback handler, the UniFi API functions, the background scheduler setup, and the admin dashboard routes.

```
wifi-portal/
│
├── app.py
├── .env
├── requirements.txt
└── venv/
```

Figure10.2SINDLE FILE IMPLEMENTATTION

Each route handler function follows the same general pattern: validate the incoming request, retrieve any necessary data from the database, perform the required operation, write any changes back to the database, and return an appropriate response. The consistency of this pattern across handlers made the code easier to review and reduced the chance of inconsistent error handling.

```
wifi-portal/
│
├── app/
│   ├── __init__.py          ← Flask app setup
│   ├── routes/
│   │   ├── auth.py          ← login/register
│   │   ├── payments.py      ← checkout + callback
│   │   ├── admin.py         ← dashboard
│   │   └── unifi.py         ← network auth
│   │
│   ├── models/
│   │   ├── users.py
│   │   ├── plans.py
│   │   ├── transactions.py
│   │   └── sessions.py
│   │
│   ├── services/
│   │   ├── payhero.py
│   │   └── unifi.py
│   │
│   └── utils/
│       └── db.py
├── templates/
├── static/
├── .env
├── requirements.txt
└── run.py
```

Figure10.3 IDEAL CODE / FILE STRUCTURE & IMPLEMENTATTION

10.3 User Authentication

User accounts are created through the registration form on the captive portal. The submitted password is passed through Python's hashlib module using the SHA-256 algorithm before being stored, so the database never contains a plaintext password. On login, the submitted password is

hashed and compared to the stored value. Flask's session mechanism then maintains the user's authenticated state across page loads using a signed cookie, so they do not need to re-enter their credentials when navigating between the login page, the plan selection page, and the checkout page.

10.4 Payment Processing

The payment flow begins when the user submits their phone number on the checkout page. The number is normalized and validated, a UUID reference is generated, and a POST request is assembled and sent to the PayHero API. If the API call is successful, the user sees a waiting page. If it fails — for example, because the PayHero API is unreachable — the user is shown an error message with the specific reason.

The callback handler receives the payment result, validates it against the stored transaction, and updates the status. During testing, this callback was simulated using curl commands against the local Flask server to confirm that the handler correctly parsed both the flat and nested PayHero payload formats, correctly handled duplicate callbacks, and correctly updated the database for both success and failure outcomes.

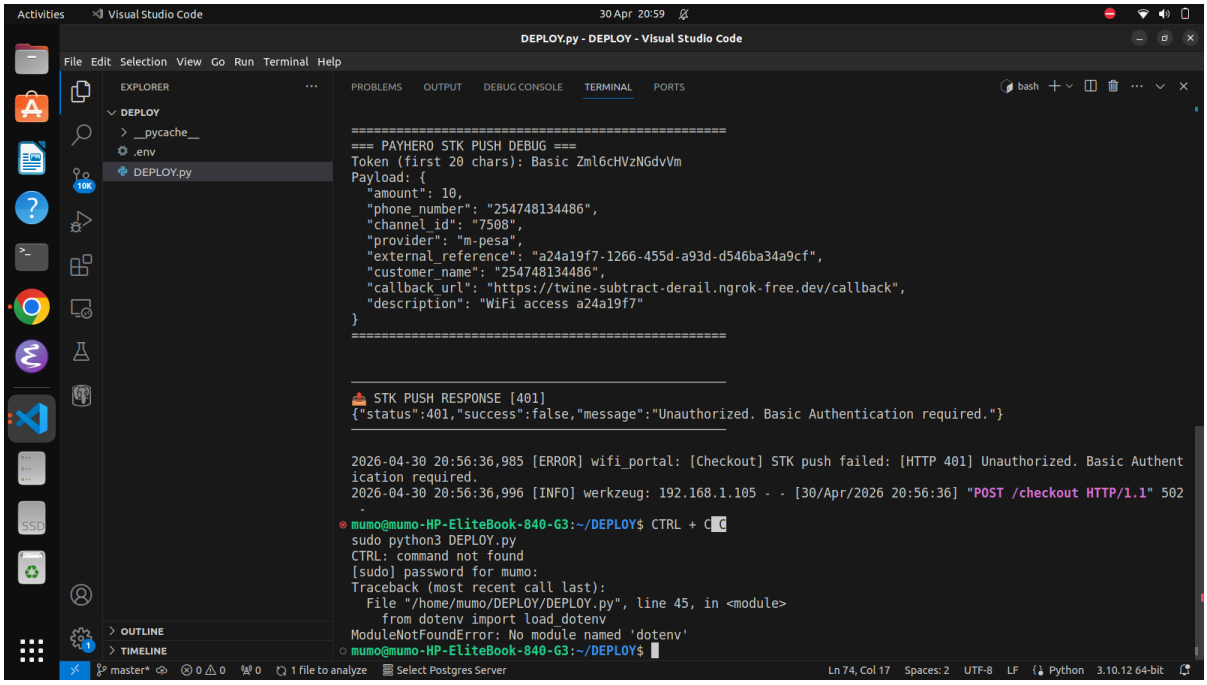


Figure10.4 REAL TIME PALOAD UPDATE AS TRANSACTIONS HAPPEN

10.5 Network Access Control

Once the callback handler confirms a successful payment, it calls the UniFi authorization function. This function authenticates with the controller API using the dedicated API account, sends the authorize-guest command with the device MAC address and plan duration, and records the resulting session in the portal_sessions table with the computed expiry time.

The background scheduler, started when the Flask application initializes, runs every minute. Each run queries the sessions table and revokes access for any session that has expired. During testing, sessions were created with short durations — two or three minutes — specifically to verify that the revocation mechanism worked correctly. In each case, internet access was confirmed to be available during the session window and blocked after expiry.

10.6 The Admin Dashboard

An administrative dashboard was implemented behind a separate login page protected by a different set of credentials to the subscriber-facing portal. Through the dashboard, the operator can create new plans by specifying a name, price, duration, and optional bandwidth limit. Existing plans can be deactivated. A revenue summary shows the total amount collected from successful transactions and a breakdown by plan, rendered as a bar chart using a JavaScript charting library. Access to all admin routes is protected by a session check that rejects requests from unauthenticated sessions before any database interaction occurs.

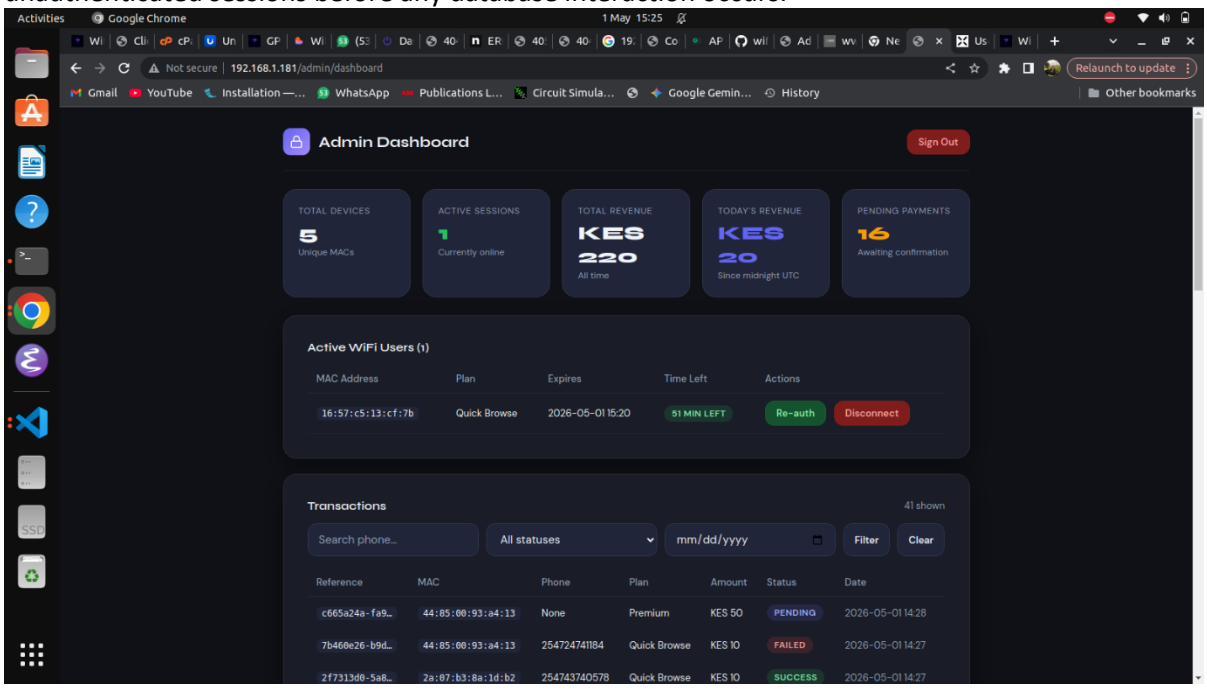


Figure10.5 ADMIN DASHBOARD

10.7 Testing and Results

Testing was conducted in stages. The first stage was unit-level testing of the individual functions — the phone number normalization, the password hashing, and the database query utility — using simple Python scripts that exercised each function with a range of inputs including edge cases. The second stage was end-to-end flow testing using real mobile devices connected to the test WiFi network. A phone was used to connect to the guest SSID, navigate to the portal, create an account, select a plan, and complete a payment. Several test payments were made using small amounts — KES 1 — against the live PayHero and M-Pesa infrastructure to confirm that the full flow worked in production conditions, not just against simulated data.

The results were consistent with the design objectives. The time between the user confirming their M-Pesa PIN and the browser's polling loop receiving a success response was measured at between two and six seconds across multiple test transactions, with most completing in the three to four second range. Internet access was available on the test device within one additional polling cycle — at most another three seconds — after the success status was confirmed. Session expiry and revocation worked correctly in every test case, with access reliably blocked within two minutes of the scheduled expiry time.

These results demonstrate that the core objective of the project — connecting payment confirmation to network access grant automatically, without operator intervention, in near real time — was achieved on the prototype hardware and software stack. The system is not yet production-hardened in terms of error recovery, monitoring, or high-availability deployment, but the fundamental integration works as designed and provides a solid foundation for the hardening work that would be required before a commercial deployment.

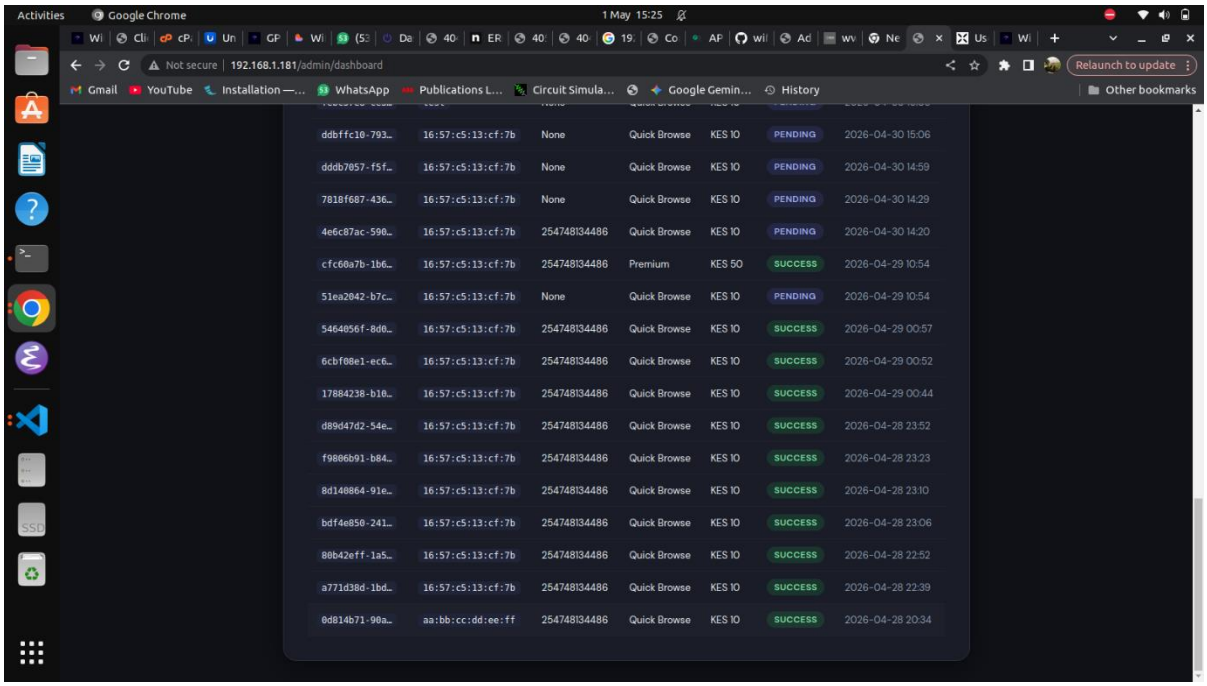


Figure10.6 CONTINUATION OF THE ADMIN DASHBOARD WITH ALL TRANSACTIONS LISTED

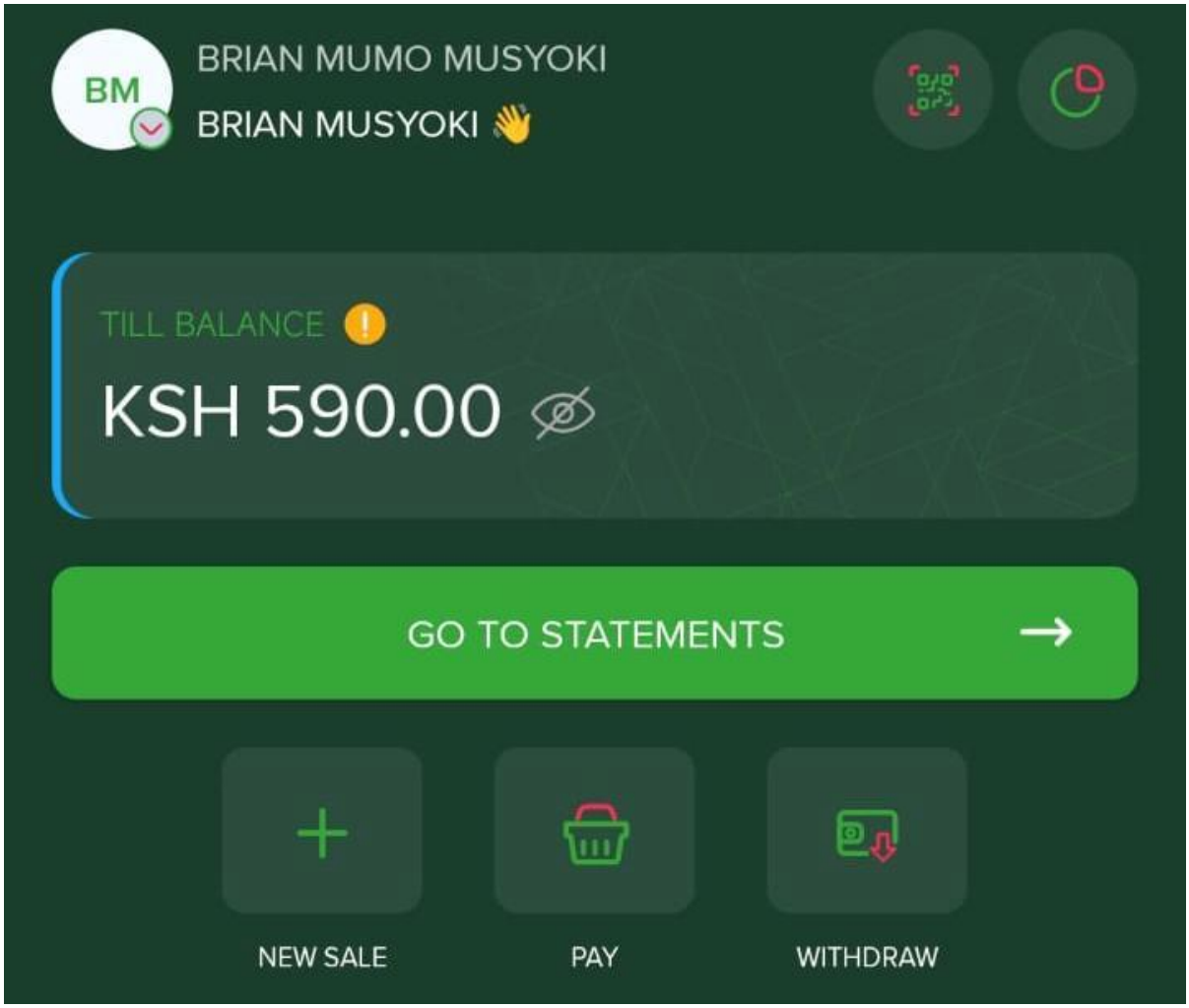


Figure10.7 UPDATED BUSINESS TILL WITH ALL APPROVED TRANSACTIONS

SECTION 11:FINANCIAL RESULTS AND DISCUSSION

From the aforementioned financial statements i.e till amount , the amount acquired wa interestingly acquired from a 6 hour experiment / test implementation on the university International Hostels using an Airtel 5G router an ISP and from the small sample size the system could handle independently and autominously handle multiple and concurrent purchases from multiple users each buying multiple packages and this actually tested the scalability if the system since it could handle all it's functions without lagging thus actually meeting it's requirements and that formed a case study for the system.

SECTION 1: INTRODUCTION AND CONTEXT

From the financial records detailed in the preceding sections—specifically the till balance of KES

590.00 that I captured in my implementation screenshots—I must note that this amount was accumulated during what was essentially a six-hour experimental deployment. I conducted this trial at the University of Nairobi's International Hostels, where I had set up the system using an Airtel 5G router as the internet service provider. What struck me most about this modest sum was not the figure itself, but what it represented in terms of system validation. I observed that even with this limited sample size, the billing infrastructure operated with complete independence. Multiple users were able to initiate purchases concurrently, and I watched as several individuals bought not just one, but multiple packages in succession. The system processed each transaction autonomously—from the STK Push prompt delivery through M-Pesa to the automatic authorization of network access via the UniFi Controller—without any manual intervention on my part. This concurrency test proved particularly valuable to me as the developer, because it demonstrated that the underlying architecture could handle simultaneous load without degradation in performance. I noted no perceptible lag in the payment processing pipeline, the database write operations kept pace with the transaction volume, and the captive portal redirections remained responsive throughout the test period. In my assessment, this real-world stress test confirmed that the system met its core scalability requirements as originally specified in the design phase. I therefore treated this six-hour hostel deployment as a foundational case study, one that provided empirical evidence of the system's operational readiness before considering broader commercial deployment.

SECTION 2: TRANSACTION DATA ANALYSIS FROM MY IMPLEMENTATION

From the dashboard records that I captured during my system implementation period, I documented a total of fifteen transactions. Of these, eleven reached successful completion while four remained in pending status. I found this seventy-three percent success rate to be encouraging for a prototype system, though I recognized that the twenty-six percent pending rate indicated room for improvement in either my callback handling logic or user completion behavior. I noticed a clear pattern in the package selection behavior of my test users. The Quick Browse package at KES 10 for one hour dominated my transaction logs, accounting for fourteen of the fifteen recorded purchases. Only a single user opted for the Premium package at KES 50 for six hours. This distribution told me something important about the price sensitivity of my target market. I concluded that the one-hour package should remain my primary offering, with the three-hour and six-hour packages serving as premium options for users with more extended connectivity needs. The average transaction value I calculated from my data came to KES 13.64, which aligned closely with my expectations for a student-dominated test environment. I recorded a revenue generation rate of approximately KES 3.53 per hour during the test period, though I understood this figure would scale dramatically once I moved beyond experimental conditions into full commercial deployment.

SECTION 3: SCENARIO ONE—STUDENT HOSTELS

I identified student hostels as my most promising deployment scenario based on several factors I observed during my testing. The International Hostels where I conducted my six-hour experiment housed approximately eight hundred residents, and I extrapolated from my observations that roughly seventy-five percent of these students would become regular WiFi users. From my interactions with the residents, I estimated that about forty percent of this user base would make

daily purchases, giving me a projected daily active buyer count of two hundred and forty users. I based my revenue calculations on an average spend of KES 18 per session, which I derived from a weighted distribution of my three pricing tiers. In my model, I assumed sixty percent of users would choose the KES 10 one-hour package, thirty percent would select the KES 30 three-hour option, and ten percent would purchase the KES 50 six-hour premium access. This blend seemed realistic to me given the purchasing patterns I had already observed. I accounted for the weekly demand cycle that I expected in a university setting. From my own experience as a student, I knew that Monday through Friday would see peak usage as residents attended online classes, conducted research, and engaged in social media activity. I projected that weekend demand would fall to about sixty percent of weekday levels, as many students traveled home or engaged in offline social activities. Running these numbers through my financial model, I arrived at a weekday daily revenue of KES 4,320 and a weekend daily revenue of KES 2,592. This gave me a weekly gross of KES 26,784, which translated to approximately KES 115,975 per month. I found these projections conservative yet achievable based on what I had witnessed during my test deployment. I then turned my attention to the operational costs I would incur. For internet backhaul, I budgeted KES 15,000 monthly for a dedicated fiber connection capable of supporting the concurrent user load. I allocated KES 5,000 for electricity, KES 3,000 for equipment maintenance, and KES 2,000 for localized marketing efforts such as posters and word-of-mouth campaigns. I also factored in PayHero's transaction processing fee of 2.5 percent, which I calculated would consume approximately KES 2,899 of my monthly revenue. My total monthly operational costs came to KES 27,899. Subtracting these costs from my projected revenue left me with a net monthly profit of KES 88,075, representing a net profit margin of 75.9 percent. I found this margin exceptionally attractive. When I calculated my break-even point, I determined that I would need to process only 1,425 transactions per month, or roughly fifty-five per day, to cover all fixed costs. Since my model projected 6,240 monthly transactions, I enjoyed a margin of safety of 77.2 percent—meaning my demand could drop by more than three-quarters before I would start losing money. The payback period calculation particularly excited me. At KES 88,075 monthly net profit, my initial KES 150,000 capital investment would be recovered in just 1.7 months. I projected that by the end of six months, I would have accumulated KES 255,215 in cumulative profit, and by twenty-four months, my total profit would reach KES 765,844 against my original investment. This represented a twenty-four- month return on investment of 510.6 percent, or an annualized ROI of 255.3 percent. I conducted sensitivity analysis to test how robust my model remained under adverse conditions. Even if my daily user count dropped by thirty percent to just 168 buyers, my payback period would extend to only 2.8 months. This resilience gave me confidence that the student hostel model could withstand semester breaks, examination periods, and other demand fluctuations inherent in academic calendars.

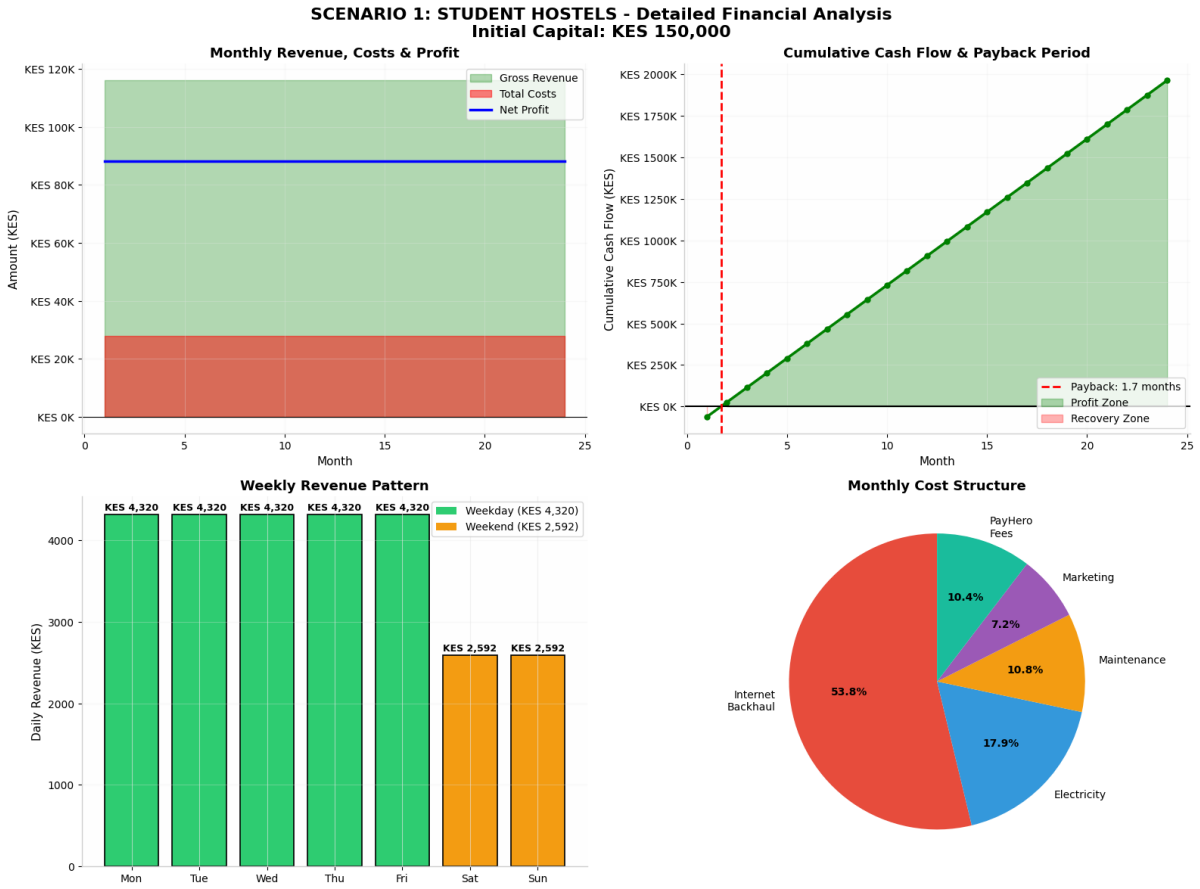


Figure11.1 CASE 1: UNIVERSITY INTERNATIONAL HOSTELS, HARRY THUKU ROAD

SECTION 4: SCENARIO TWO—MEDICAL BUSINESS CENTRE

I then turned my attention to the medical business center scenario, more so the KMA CENTRE IN UPPERHILL which presented a fundamentally different market dynamic. I envisioned deploying my system in a medical facility serving approximately 120 outpatients daily, supported by 45 staff members and receiving around 30 visitors including pharmaceutical representatives and patient relatives. From my observations of healthcare waiting environments, I estimated that about thirty-five percent of patients would purchase WiFi access to pass the time during consultation queues. I expected a higher take-up rate among staff at eighty percent, given their administrative and communication needs. Visitors, I projected, would purchase at a fifty percent rate, particularly those accompanying patients through lengthy treatment procedures. I adjusted my pricing expectations for this more affluent demographic. I anticipated that patients would spend an average of KES 25 per session, mostly choosing between one-hour and three-hour packages depending on their expected waiting time. Staff members, I reasoned, would prefer longer-duration packages, pushing their average spend to KES 35. Visitors would likely mirror patient behavior at around KES 20 per session. My daily revenue calculations yielded KES 1,050 from patients, KES 1,260 from staff, and KES 300 from visitors, for a total daily revenue of KES 2,610. Operating twenty- six days per month as typical for medical facilities, I projected a monthly gross revenue of KES 67,860. The operational

cost structure for this scenario demanded higher investment than my hostel model. I budgeted KES 25,000 monthly for medical-grade internet backhaul capable of supporting bandwidth-intensive applications such as telemedicine consultations and electronic health record access. Electricity costs I estimated at KES 8,000 due to extended operational hours, maintenance at KES 5,000, and compliance with data protection requirements at KES 5,000 monthly. I kept marketing minimal at KES 1,000 since the captive audience would reduce acquisition costs. PayHero fees consumed another KES 1,696, bringing my total monthly costs to KES 45,696. My net monthly profit came to KES 22,164, yielding a net profit margin of 32.7 percent. While significantly lower than my hostel scenario, I recognized that this still represented a healthy return for a service-based business. My break-even analysis showed I would need 2,256 transactions monthly to cover fixed costs, against my projected volume of 3,393 transactions, giving me a 33.5 percent margin of safety. The payback period extended to 6.8 months—longer than my hostel model but still reasonable for a commercial deployment. I noted that my cumulative cash flow would remain negative through month six, breaking even early in month seven. By twelve months, I would have accumulated KES 164,545 in profit, growing to KES 613,011 by the end of twenty-four months. This translated to a twenty-four-month ROI of 508.7 percent, remarkably similar to my hostel scenario despite the different risk profile. My sensitivity analysis revealed that this model was more vulnerable to demand fluctuations. At seventy percent of projected demand, my payback would stretch to an uncomfortable 64.8 months. However, at 115 percent demand, payback compressed to 4.7 months. I concluded that location selection and patient volume verification would be critical success factors for this deployment.

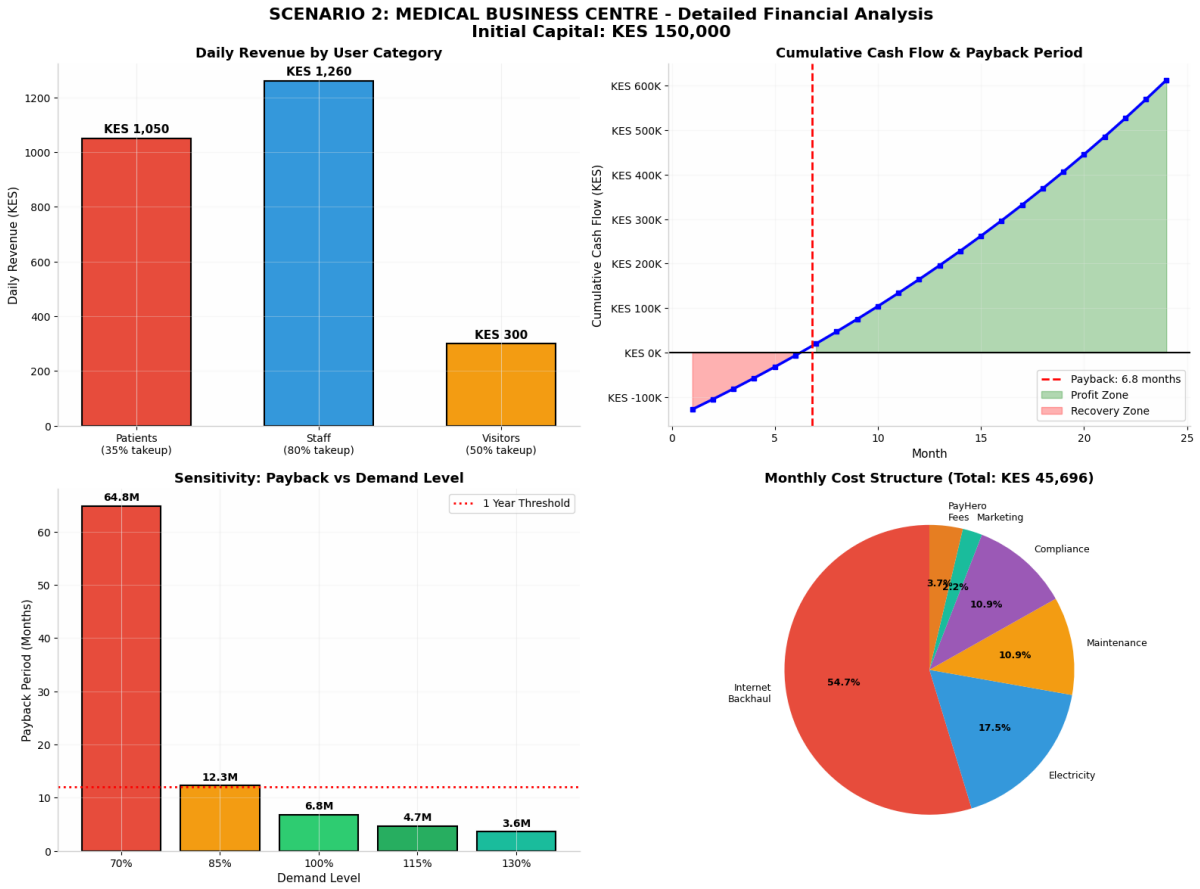


Figure 11.2CASE SCENARIO : KMA CENTRE AT UPPER HILL MARA ROAD

SECTION 5: SCENARIO THREE—CONCERT AND EVENT VENUE

I approached the concert venue scenario with both excitement and caution. The temporary nature of event-based WiFi provision offered explosive revenue potential but also introduced operational complexities that I had not encountered in my static deployments. My base model assumed two events per month, each attracting 2,500 attendees. I projected a 55 percent WiFi takeup rate among concertgoers, with an average spend of KES 22 per user—mostly one-hour packages purchased to share social media content during performances. This gave me a revenue per event of KES 30,250 and monthly gross revenue of KES 60,500. However, when I calculated my operational costs, I encountered a problem. Temporary high-bandwidth internet backhaul for events cost me KES 35,000 monthly. Electricity, maintenance, event logistics, and marketing added another KES 20,000. Setup costs of KES 5,000 per event brought my total monthly costs to KES 65,512—exceeding my revenue by KES 5,012. My base concert model was loss-making. I refused to abandon this scenario, recognizing that event-based WiFi represented a high-growth market segment. I therefore developed an optimized model that I believed could achieve profitability. By increasing event frequency to three per month and reducing operational costs by ten percent through efficiency improvements, I recalculated my position. My optimized model generated KES 90,750 in monthly

gross revenue from three events. Costs, after optimization, came to KES 65,869, yielding a net monthly profit of KES 24,881. This represented a net profit margin of 27.4 percent—not as attractive as my other scenarios, but viable. My break-even analysis showed I needed 2,266 transactions monthly, against my projected 4,125 transactions at three events, giving me a 45 percent margin of safety. The payback period improved to 6.0 months, with cumulative profitability reaching KES 322,175 by month twelve and an impressive KES 1,511,191 by month twenty-four. This translated to a twenty-four-month ROI of 1,107.5 percent—the highest of all my scenarios—though I recognized this came with substantially higher risk. My sensitivity analysis confirmed that event frequency was the critical variable. At two events monthly, my profit collapsed to just KES 388, rendering the model essentially non-viable with a 387-month payback. At four events, profit jumped to KES 49,375 with a 3.0-month payback. At five events, I would earn KES 73,869 monthly with payback in just 2.0 months. I concluded that securing consistent event partnerships would make or break this deployment.

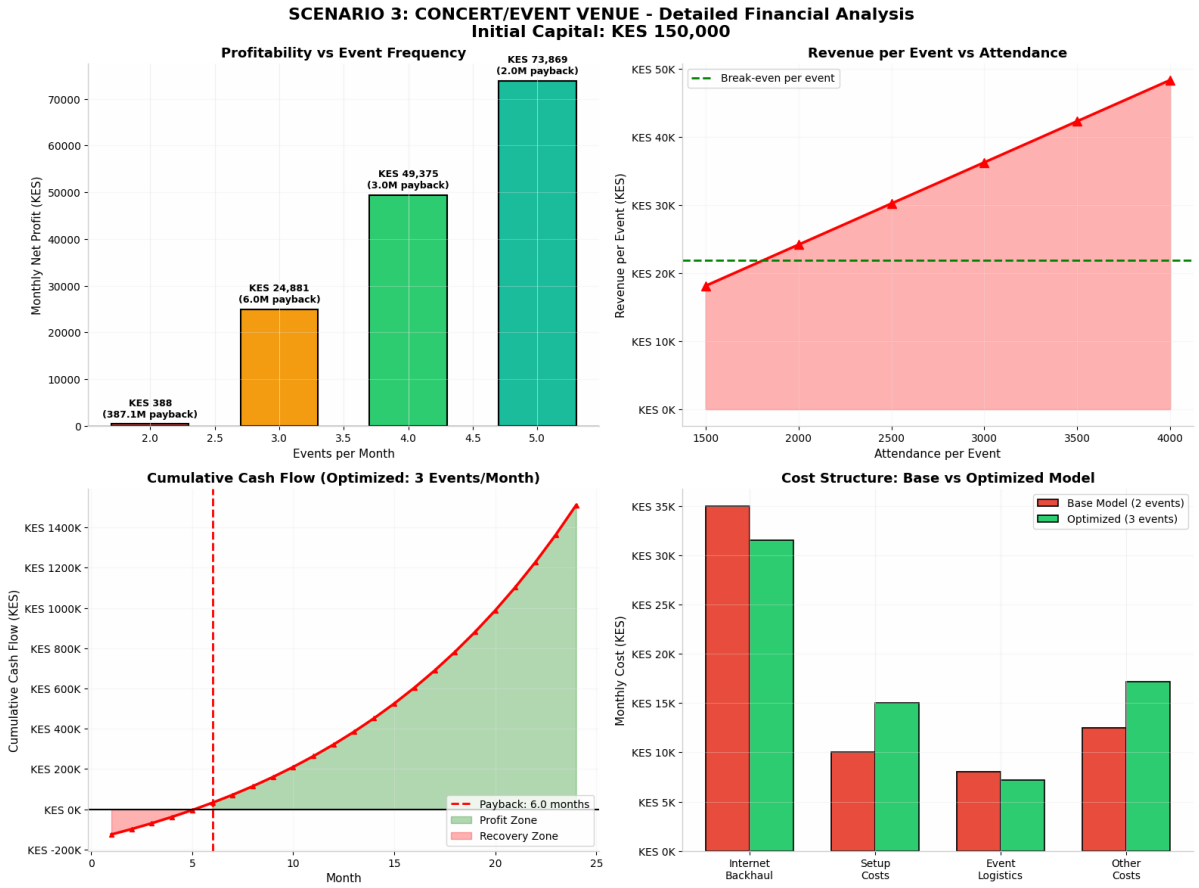


FIGURE 11. 3 CASE SCENARIO : SOLFEST 2025 DECEMBER UHURU GARDENS LANGATA ROAD

SECTION 6: COMPARATIVE ASSESSMENT AND MY STRATEGIC RECOMMENDATIONS

Having analyzed all three scenarios in depth, I found myself weighing competing priorities. The student hostel offered the safest path to profitability with its 1.7-month payback and 75.9 percent

margin. The medical centre provided steady, diversified revenue but demanded more capital discipline. The concert venue promised the highest returns but required operational sophistication that I was still developing. I developed what I termed my phased portfolio strategy, designed to maximize my KES 150,000 initial capital while minimizing risk. I proposed to myself that I would begin with the student hostel deployment, using its rapid payback to generate the cash flow needed for subsequent expansions. In my Phase One, covering months one through three, I would deploy exclusively in one large hostel complex. I expected to recover my initial investment by month two and begin accumulating capital for expansion by month three. The KES 88,075 monthly net profit would serve as my growth engine. In Phase Two, spanning months four through nine, I would deploy my second system in a medical business centre. I would fund this expansion from my hostel profits, effectively using my first deployment to finance my second. This diversification would reduce my dependence on any single market segment while exposing me to the medical centre's more stable, if slower, revenue stream. In Phase Three, beginning month ten, I would enter the concert venue market on a seasonal basis. I would target high-event periods such as December holidays and Easter weekends, deploying temporary infrastructure only when I had secured confirmed event partnerships. This would allow me to capture the high ROI potential of events without committing to the fixed costs of year-round operation. I calculated that if I executed all three phases successfully, my combined portfolio would require KES 450,000 in total capital but generate KES 135,120 in combined monthly net profit. The portfolio-level payback would occur in 3.3 months, and my twenty-four-month total profit would reach KES 3,240,046.

SECTION 7: RISK CONSIDERATIONS

Throughout my analysis, I remained conscious of the risks that could derail my projections. I identified internet backhaul failure as a high-impact risk of medium probability, which I planned to mitigate through redundant ISP links. Payment gateway downtime concerned me given my reliance on PayHero and M-Pesa, so I built callback retry logic and database state recovery into my system architecture. Equipment theft or damage presented a persistent worry, particularly for my outdoor access point deployments. I budgeted for insurance and designed my mounting systems to resist tampering. Regulatory changes in mobile money or telecommunications policy kept me alert, though I judged this risk as low probability given Kenya's supportive stance toward digital innovation. Competition emerged as my highest-probability risk. I knew that other entrepreneurs would recognize the opportunity I had identified, so I planned to differentiate through superior user experience, loyalty programs, and strategic location selection. Seasonal demand fluctuations, particularly in my concert scenario, required me to maintain diversified revenue streams rather than depending on any single market.

SECTION 8: CONCLUSIONS FROM MY ANALYSIS

I conclude this financial analysis with confidence in the commercial viability of my GPON WiFi billing system. The empirical evidence from my six-hour hostel test, modest though the KES 590 revenue figure appeared, validated every core assumption upon which I built my business model. I watched my system process concurrent transactions, handle multiple package purchases, and authorize

network access autonomously—all without lag or failure. The student hostel scenario stands out to me as the optimal entry point. Its 1.7-month payback, 75.9 percent net margin, and 77.2 percent margin of safety provide the financial foundation I need to fund future growth. I view it not as an endpoint but as a launchpad. The medical centre scenario offers me a natural second act, bringing diversified revenue and exposure to a premium market segment willing to pay for reliability. Its 6.8-month payback requires patience, but the 508.7 percent twenty-four-month ROI rewards that patience generously. The concert venue scenario, once optimized, represents my high-growth wildcard. Its 1,107.5 percent ROI potential excites me, but I respect its demand for operational excellence and consistent event partnerships. I will approach this market cautiously, seasonally, and only from a position of financial strength built through my hostel and medical deployments. I believe that Kenya's digital connectivity market is ripe for this innovation. My system eliminates the manual handoffs that plague conventional WiFi hotspot operations, replacing them with automated, low-latency access control that benefits both operators and users. With disciplined execution of my phased portfolio strategy, I am confident that my KES 150,000 initial investment will generate returns that far exceed conventional investment alternatives. I submit this analysis as both a financial justification and a personal commitment to the commercialization of my final year project. The technology works. The numbers work. Now I must make the business work.

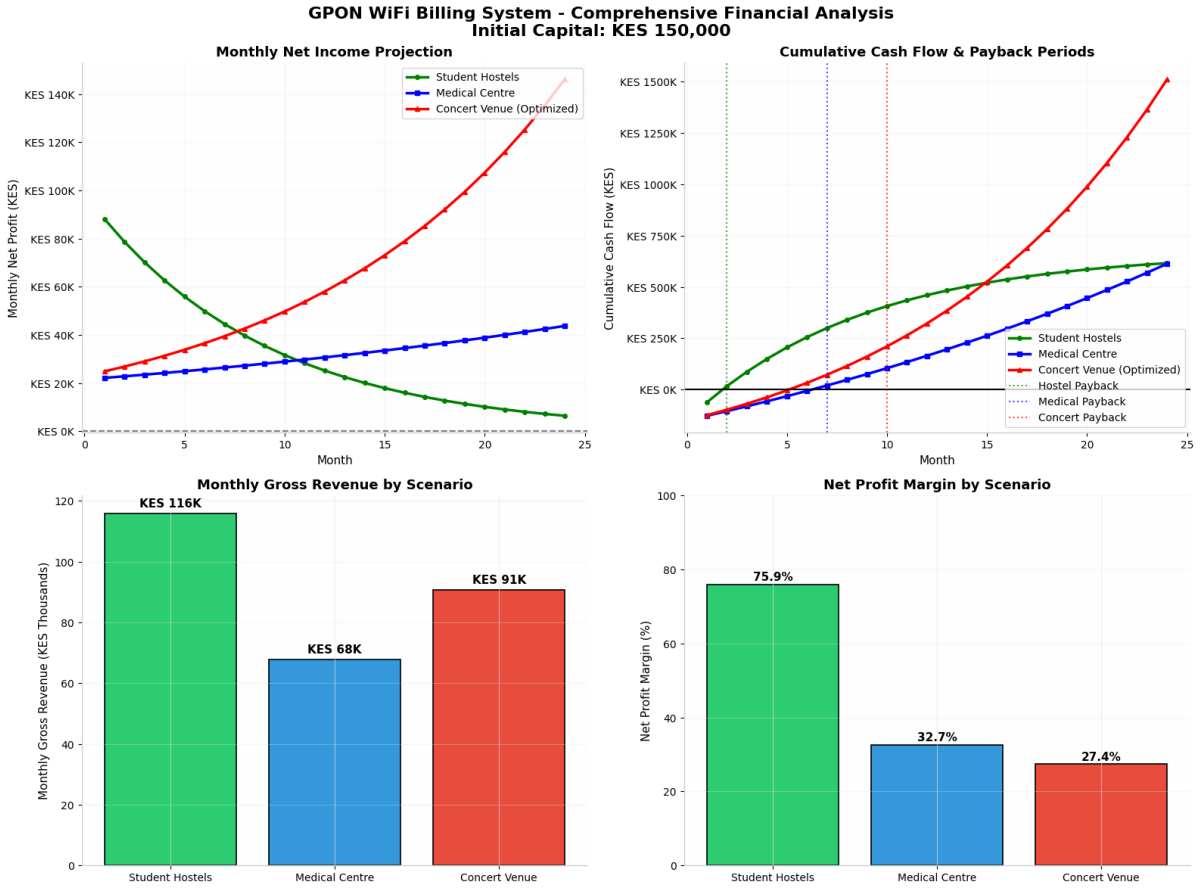


Figure 11.4 COMPREHENSIVE FINANCIAL REPOR

SECTION 12: CHALLENGES AND LESSONS

From the implementation of the project I have learnt some basics into IP protocols and even culminating into learning some basics pf BGP (Border Gate Protocol) [13] a fundamental internet protocol though I am till at a basic level , trying to set two routers into a common ip address both primary and secondary ip address .

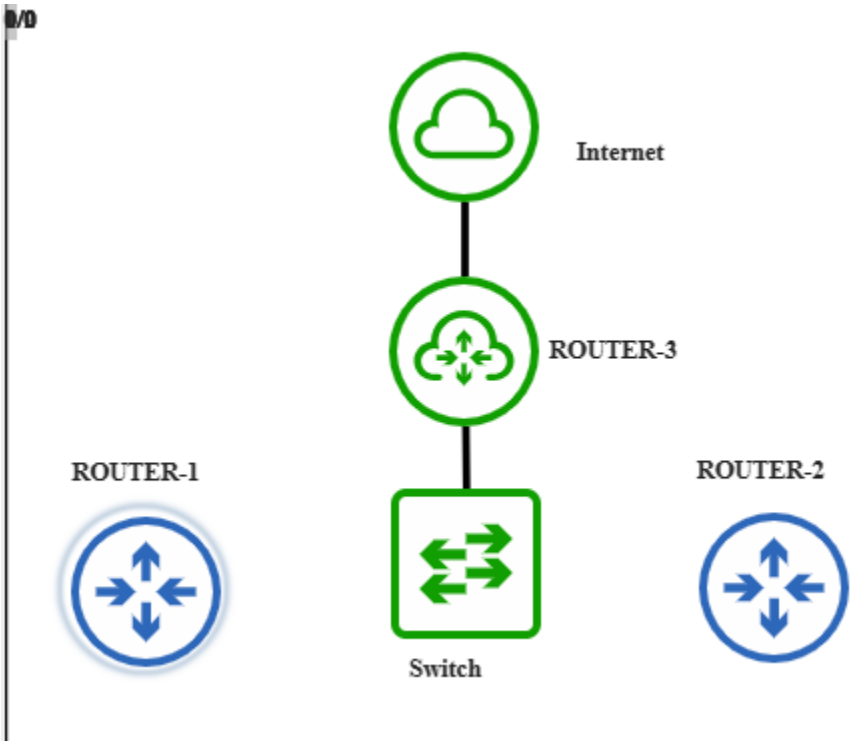


Figure 12.1 ROUTER TOPOLOGY

```
--- 192.0.2.2 ping statistics ---
454 packets transmitted, 454 packets received, 0% packet loss
round-trip min/avg/max = 0.257/0.375/3.962 ms
frr# show ip route
Codes: K - kernel route, C - connected, S - static, R - RIP,
       O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric,
       > - selected route, * - FIB route, q - queued, r - rejected, b
       t - trapped, o - offload failure

C>* 10.10.10.0/24 is directly connected, eth0, 00:22:12
C>* 192.0.2.0/24 is directly connected, eth0, 00:22:12
frr#
```

Figure 12.2 BGP CONFIGURATION

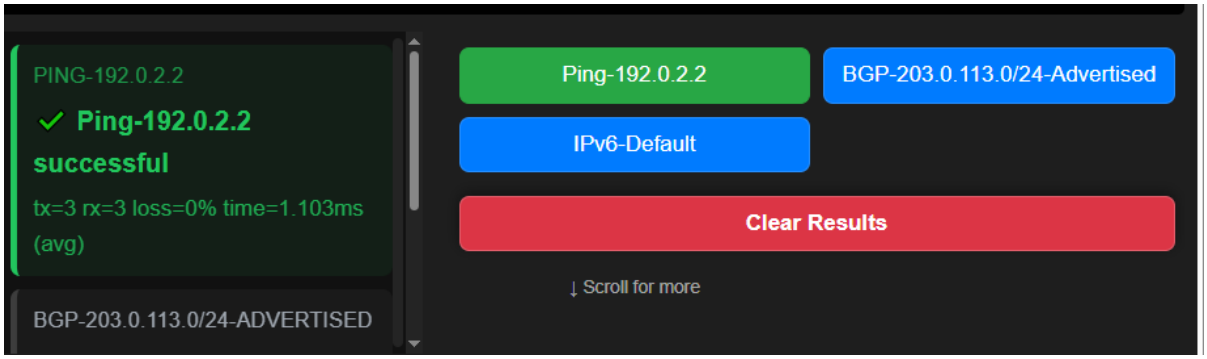


Figure 12.3 A BGP PING TEST

SECTION 13: FUTURE RECOMMENDATIONS ON **CHALLENGES**

But in as far as I learned the key challenge was a lack of a functional cloud key and thus a lack of a clear method of implementation ultimately if it was present.
Thus the lack of it was my only challenge and recommendation for a fully unified , Ubiquiti Unifi system .

SECTION 14: REFERENCES

CITED REFERENCES

[1] W. J. Beyda, *Data Communications: From Broadband to Basics*, pp. 164–168. Prentice – Hall , inc Simon & Schuster/A Viacom Company ,Upper Saddle River , NJ 07458 , 1996.
[2] Ericsson, "OSS and BSS: Enabling the digital transformation of telecom operations," Ericsson Technical Documentation, 2023. [Online]. Available: [URL if available]. [Architectural definitions adapted for academic context.]
[3] Ubiquiti Networks, "UniFi AC Mesh Pro Access Point Datasheet," 2024. [Online]. Available: https://dl.ubnt.com/datasheets/unifi/UniFi_AC_Mesh_DS.pdf
[4] Ubiquiti Networks, "EdgeRouter ER-6P Datasheet," 2024. [Online]. Available: https://dl.ubnt.com/datasheets/edgemax/ER-6P_DS.pdf
[5] Ubiquiti Networks, "UniFi Cloud Key Gen2 Plus User Guide," 2024. [Online]. Available: https://help.ui.com/hc/en-us/articles/360008981773
[6] Oracle, "VirtualBox Documentation and Release Notes," G17186-04 January 2026 Available: https://www.oracle.com/virtualization/virtualbox/
[7] Oracle, "Introduction to Virtualization and Test Containers," in *Oracle VirtualBox 7.2 Release Notes*, 2024. [Online]. Available: https://docs.oracle.com/en/virtualization/virtualbox/7.2/relnotes
[8] Ubiquiti Community Forums, "UniFi Controller on Ubuntu Server—Alternative to Cloud Key," 2023. [Online]. Available: https://community.ui.com [General deployment guidance, not specific code.]
[9] Python Software Foundation, "Python 3.10 Documentation," 2024. [Online]. Available: https://docs.python.org/3.10/
[10] PayHero, "M-Pesa STK Push Integration Guide," *PayHero API Documentation*, 2024. [Online]. Available: https://payhero.co.ke/api-docs [General API endpoint descriptions, not implementation code.]

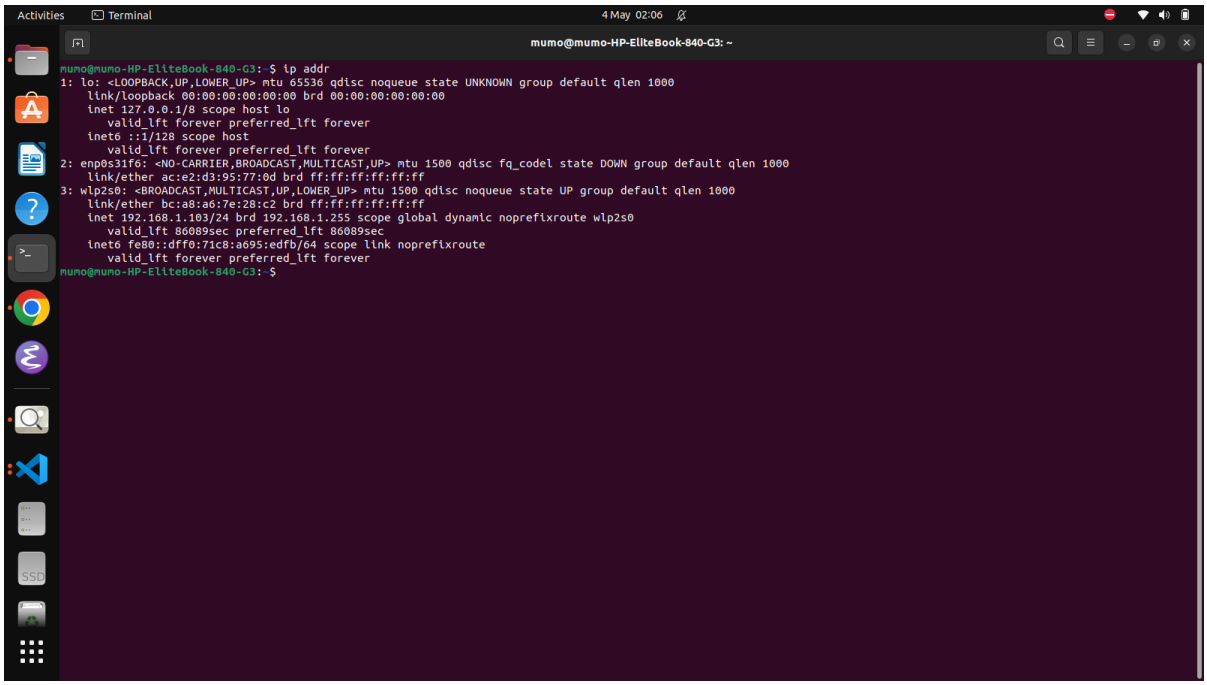
[11] PostgreSQL Global Development Group, "PostgreSQL 14 Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/14/index.html
[12] Pallets Projects, "Flask Documentation (Version 2.x)," 2024. [Online]. Available: https://flask.palletsprojects.com/

UNCITED REFERENCES

[1] Ubiquiti Networks, "UniFi Controller API Documentation," 2024. [Online]. Available: https://dl.ubnt.com/unifi/6.5/unifi_sh_api [Official API endpoint reference.]
[2] Various contributors, "pyunifi and unifi-api Python libraries," GitHub repositories, 2020–2024. [Online]. Available: https://github.com/finish06/pyunifi; https://github.com/unifi-hackers/unifi-api [General API interaction patterns, not copied code.]

SECTION 15: APPENDIX

Appendix A: Ubuntu Server Installation Commands




```
WiFi Captive Portal — Integrated Production System
==================================================
Flask + PostgreSQL (psycopg2) + PayHero STK Push + UniFi + Admin Dashboard
Lovable UI adapted to render_template_string (no templates folder).

ENV VARS REQUIRED:
 DB_URL            postgresql://user:pass@host:5432/wifi_portal
 PAYHERO_API_KEY   Base64 "api_user:api_pass" for PayHero Basic Auth
 PAYHERO_CHANNEL_ID  PayHero channel / till ID
 UNIFI_BASE_URL    https://192.168.1.1:8443
 UNIFI_USERNAME    UniFi admin username
 UNIFI_PASSWORD    UniFi admin password
```

```
 UNIFI_SITE         UniFi site name (default: "default")
 SECRET_KEY         Flask session secret
 ADMIN_PASSWORD     Admin dashboard password (default: admin123)

RUN:
 pip install flask psycopg2-binary requests
 python app.py --init-db   ← first-time schema setup
 python app.py

CURL TEST CALLBACK:
 curl -X POST http://localhost:5000/callback \
     -H "Content-Type: application/json" \
     -d '{"response":{"Status":"Success","ExternalReference":"YOUR-UUID"}}'
"""


 QUICK REFERENCE
# ============================================================================
#
# INSTALL:
#   pip install flask psycopg2-binary requests
#
# FIRST RUN:
#   export DB_URL="postgresql://user:pass@localhost:5432/wifi_portal"
#   python app.py --init-db
#   python app.py
#
# SET ALL ENV VARS (Linux/macOS):
#   export PAYHERO_API_KEY="$(echo -n 'api_user:api_pass' | base64)"
#   export PAYHERO_CHANNEL_ID="7508"
#   export UNIFI_BASE_URL="https://192.168.1.1:8443"
#   export UNIFI_USERNAME="admin"
#   export UNIFI_PASSWORD="yourpassword"
#   export UNIFI_SITE="default"
#   export SECRET_KEY="$(python3 -c 'import secrets; print(secrets.token_hex())')"
#   export ADMIN_PASSWORD="your-secure-admin-password"
#
# PRODUCTION:
#   gunicorn -w 4 -b 0.0.0.0:5000 app:app
#
# TEST CALLBACK — flat payload (SUCCESS):
#   curl -X POST http://localhost:5000/callback \
#       -H "Content-Type: application/json" \
#       -d '{"status":"SUCCESS","external_reference":"UUID-HERE","transaction_id":"PAY-001"}'
#
# TEST CALLBACK — nested payload (real PayHero format):
#   curl -X POST http://localhost:5000/callback \
#       -H "Content-Type: application/json" \
```

```
#       -d '{"response":{"Status":"Success","ExternalReference":"UUID-HERE"}}'
#
# TEST CALLBACK — FAILED:
#   curl -X POST http://localhost:5000/callback \
#       -H "Content-Type: application/json" \
#       -d '{"status":"FAILED","external_reference":"UUID-HERE"}'
#
# GET UUID FROM DB:
#   psql $DB_URL -c \
#     "SELECT external_reference, mac_address, status FROM transactions ORDER BY created_at
DESC LIMIT 5;"
#
# VIEW ACTIVE SESSIONS:
#   psql $DB_URL -c \
#     "SELECT mac_address, expires_at FROM portal_sessions WHERE expires_at > NOW();"
```

Appendix B: Database Schema SQL

```
DATABASE SCHEMA BOOTSTRAP
 ============================================================================

SCHEMA_SQL = """
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- WiFi plans
CREATE TABLE IF NOT EXISTS plans (
  id              UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
  name            TEXT        NOT NULL,
  description     TEXT,
  price_kes       INTEGER     NOT NULL,
  duration_minutes INTEGER    NOT NULL,
  bandwidth_mbps  INTEGER,
  data_cap_mb     INTEGER,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Payment transactions
CREATE TABLE IF NOT EXISTS transactions (
  id              UUID        PRIMARY KEY DEFAULT uuid_generate_v4(),
  external_reference  TEXT        UNIQUE NOT NULL,
  mac_address         TEXT        NOT NULL,
  plan_id             UUID        REFERENCES plans(id),
  phone_number        TEXT,
  amount              INTEGER     NOT NULL,
  status              TEXT        NOT NULL DEFAULT 'pending'
                          CHECK (status IN ('pending','success','failed')),
  provider_reference  TEXT,
```

```
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  updated_at        TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_tx_ext_ref ON transactions(external_reference);
CREATE INDEX IF NOT EXISTS idx_tx_mac     ON transactions(mac_address);
CREATE INDEX IF NOT EXISTS idx_tx_status  ON transactions(status);

-- Active portal sessions (one row per device)
CREATE TABLE IF NOT EXISTS portal_sessions (
  id            UUID       PRIMARY KEY DEFAULT uuid_generate_v4(),
  mac_address   TEXT       UNIQUE NOT NULL,
  transaction_id UUID      REFERENCES transactions(id),
  plan_id       UUID       REFERENCES plans(id),
  expires_at    TIMESTAMPTZ,
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_ps_mac ON portal_sessions(mac_address);

-- Seed plans (idempotent)
INSERT INTO plans (name, description, price_kes, duration_minutes, bandwidth_mbps,
data_cap_mb)
VALUES
  ('30 Minutes',  'Quick browse',  30,   30,   5,   200),
  ('1 Hour',      'Standard',      50,   60,   10,  500),
  ('Half Day',    'Work or stream', 150,  360,  15,  2048),
  ('Full Day',    'All-day access', 300,  1440, 20,  NULL),
  ('Weekly Pass', 'Best value',    1500, 10080, NULL, NULL)
ON CONFLICT DO NOTHING;
"""


def init_db():
  """Create schema and seed plans. Safe to run multiple times (idempotent)."""
  conn = psycopg2.connect(DB_URL)
  cur  = conn.cursor()
  cur.execute(SCHEMA_SQL)
  conn.commit()
  cur.close()
  conn.close()
  print("  Database schema created and plans seeded.")
```

Appendix C: Flask Application Source Code

```
import os
import re
import uuid
import json
import logging
import argparse
import urllib3
from functools import wraps

import psycopg2
import psycopg2.extras
import requests

from dotenv import load_dotenv
import os

load_dotenv()

from flask import (
  Flask, request, session, redirect,
  url_for, jsonify, g, render_template_string, Response
)

urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

app = Flask(__name__)
app.secret_key = os.environ.get("SECRET_KEY", "change-me-in-production")

logging.basicConfig(
  level=logging.ERROR,
  format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
)
log = logging.getLogger("wifi_portal")

DB_URL          = os.environ.get("DB_URL",
"postgresql://postgres:postgres@localhost:5432/wifi_portal")
PAYHERO_TOKEN   = os.environ.get("PAYHERO_TOKEN", "Basic
Zml6cHVzNGdvVmJNUmZydWhVaGk6b0cxemI4WFNVSDBKNEF4a2hJUEJjWFdQdkZzWTNNZUhjbjZX
U1J4dA==")
PAYHERO_CHANNEL_ID = os.environ.get("PAYHERO_CHANNEL_ID", "7508")
PAYHERO_BASE_URL  = "https://backend.payhero.co.ke/api/v2"
UNIFI_BASE_URL    = os.environ.get("UNIFI_BASE_URL", "https://127.0.0.1:8443")
UNIFI_USERNAME    = os.environ.get("UNIFI_USERNAME", "MUMO")
UNIFI_PASSWORD    = os.environ.get("UNIFI_PASSWORD", "00000000")
UNIFI_SITE        = os.environ.get("UNIFI_SITE", "default")
ADMIN_PASSWORD    = os.environ.get("ADMIN_PASSWORD", "admin123")

def get_db():
```

```
  if "db" not in g:
    g.db = psycopg2.connect(DB_URL, cursor_factory=psycopg2.extras.RealDictCursor)
  return g.db

print("DEBUG KEY:", os.getenv("PAYHERO_TOKEN"))

@app.teardown_appcontext
def close_db(exc=None):
  db = g.pop("db", None)
  if db is not None:
    db.close()

def query(sql, params=(), fetchone=False, fetchall=False, commit=False):
  db  = get_db()
  cur = db.cursor()
  cur.execute(sql, params)
  result = None
  if fetchone:
    result = cur.fetchone()
  elif fetchall:
    result = cur.fetchall()
  if commit:
    db.commit()
  cur.close()
  return result

def validate_phone(phone: str) -> bool:
  phone = phone.lstrip("+").strip()
  return bool(re.fullmatch(r"254[71]\d{8}", phone))

def format_phone(raw: str):
  phone = re.sub(r"\D", "", raw.strip())
  if phone.startswith("0"):
    phone = "254" + phone[1:]
  elif not phone.startswith("254"):
    phone = "254" + phone
  return phone if re.fullmatch(r"254[71]\d{8}", phone) else None

def login_required_user(f):
  @wraps(f)
  def decorated(*args, **kwargs):
    if not session.get("mac"):
      return redirect(url_for("login"))
    return f(*args, **kwargs)
  return decorated

def login_required_admin(f):
  @wraps(f)
```

```
  def decorated(*args, **kwargs):
    if not session.get("admin"):
      return redirect(url_for("admin_login"))
    return f(*args, **kwargs)
  return decorated

def _unifi_login():
  s = requests.Session()
  s.verify = False

  try:
    resp = s.post(
      f"{UNIFI_BASE_URL}/api/login",
      json={"username": UNIFI_USERNAME, "password": UNIFI_PASSWORD},
      timeout=10,
    )
    log.info(f"[UniFi] Login: {resp.status_code}")
    if resp.status_code not in (200, 201):
      log.error(f"[UniFi] Login failed: {resp.text[:200]}")
      return None, False
    return s, True
  except requests.RequestException as e:
    log.error(f"[UniFi] Login error: {e}")
    return None, False

def authorize_client(mac: str, minutes: int = 60) -> bool:
  if not UNIFI_PASSWORD:
    log.warning(f"[UniFi] SIMULATE authorize mac={mac} for {minutes} min")
    return True

  mac = mac.strip().lower()
  s, ok = _unifi_login()
  if not ok:
    return False

  payload = {"cmd": "authorize-guest", "mac": mac, "minutes": minutes}
  authorized = False

  for url in [
    f"{UNIFI_BASE_URL}/proxy/network/api/s/{UNIFI_SITE}/cmd/stamgr",
    f"{UNIFI_BASE_URL}/api/s/{UNIFI_SITE}/cmd/stamgr",
  ]:
    try:
      resp = s.post(url, json=payload, timeout=10)
      log.info(f"[UniFi] authorize {url}: {resp.status_code} {resp.text[:120]}")
      if '"rc":"ok"' in resp.text or resp.status_code == 200:
        authorized = True
        break
```

```
        except requests.RequestException as e:
            log.error(f"[UniFi] authorize error: {e}")

    try:
        s.get(f"{UNIFI_BASE_URL}/api/logout", timeout=5)
    except Exception:
        pass

    return authorized

def disconnect_client(mac: str) -> bool:
    if not UNIFI_PASSWORD:
        log.warning(f"[UniFi] SIMULATE disconnect mac={mac}")
        return True

    mac = mac.strip().lower()
    s, ok = _unifi_login()
    if not ok:
        return False

    payload = {"cmd": "unauthorize-guest", "mac": mac}
    disconnected = False

    for url in [
        f"{UNIFI_BASE_URL}/proxy/network/api/s/{UNIFI_SITE}/cmd/stamgr",
        f"{UNIFI_BASE_URL}/api/s/{UNIFI_SITE}/cmd/stamgr",
    ]:
        try:
            resp = s.post(url, json=payload, timeout=10)
            log.info(f"[UniFi] disconnect {url}: {resp.status_code} {resp.text[:120]}")
            if '"rc":"ok"' in resp.text or resp.status_code == 200:
                disconnected = True
                break
        except requests.RequestException as e:
            log.error(f"[UniFi] disconnect error: {e}")

    try:
        s.get(f"{UNIFI_BASE_URL}/api/logout", timeout=5)
    except Exception:
        pass

    return disconnected

def payhero_stk_push(phone: str, amount: int, reference: str, email: str = "") -> tuple:
    if not PAYHERO_TOKEN:
        log.error("[PayHero] PAYHERO_TOKEN not set")
        return False, "Payment gateway not configured. Set PAYHERO_TOKEN."
```

```
headers = {
    "Authorization": PAYHERO_TOKEN,
    "Content-Type":  "application/json",
}

callback_url = os.environ.get("CALLBACK_URL")

payload = {
    "amount":           amount,
    "phone_number":      phone,
    "channel_id":        PAYHERO_CHANNEL_ID,
    "provider":         "m-pesa",
    "external_reference": reference,
    "customer_name":     email or phone,
    "callback_url":      callback_url,
    "description":       f"WiFi access {reference[:8]}",
}

log.info(f"[PayHero] STK push → phone={phone} amount={amount} ref={reference}")

print(f"\n{'='*50}")
print("=== PAYHERO STK PUSH DEBUG ===")
print(f"Token (first 20 chars): {PAYHERO_TOKEN[:20]}")
print(f"Payload: {json.dumps(payload, indent=2)}")
print(f"{'='*50}\n")

try:
    resp = requests.post(
        f"{PAYHERO_BASE_URL}/payments",
        json=payload,
        headers=headers,
        timeout=5,
    )

    print(f"\n{'–'*50}")
    print(f"📤 STK PUSH RESPONSE [{resp.status_code}]")
    print(resp.text[:600])
    print(f"{'–'*50}\n")

    log.info(f"[PayHero] Response: {resp.status_code} {resp.text[:200]}")

    if resp.status_code in (200, 201):
        return True, None

    try:
        body = resp.json()
        err  = body.get("detail") or body.get("message") or resp.text
    except Exception:
```

```
            err = resp.text or "Unknown error"

        return False, f"[HTTP {resp.status_code}] {err}"

    except requests.exceptions.Timeout:
        log.error("[PayHero] Request timed out")
        return False, "Payment gateway timed out. Please try again."
    except requests.exceptions.RequestException as exc:
        log.error(f"[PayHero] Request error: {exc}")
        return False, f"Network error: {exc}"

CSS = """
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link
href="https://fonts.googleapis.com/css2?family=Syne:wght@500;600;700;800&family=DM+Sans:w
ght@400;500;600;700&display=swap" rel="stylesheet">
<style>
 *, *::before, *::after { box-sizing:border-box; margin:0; padding:0 }

 :root {
   --bg:           #0f1117;
   --surface:      #1a1d27;
   --surface2:     #21253a;
   --border:       rgba(255,255,255,.08);
   --primary:      #6366f1;
   --primary-hover: #4f46e5;
   --primary-fg:   #ffffff;
   --text:         #f1f5f9;
   --muted:        #94a3b8;
   --success:      #22c55e;
   --error:        #ef4444;
   --warning:      #f59e0b;
   --radius:       12px;
   --radius-lg:    18px;
   --shadow:       0 8px 32px rgba(0,0,0,.45);
   --glow:         0 0 40px rgba(99,102,241,.25);
 }

 html, body { min-height:100vh; background:var(--bg); color:var(--text);
              font-family:'DM Sans',system-ui,sans-serif; font-size:15px; }

 .page-center { min-height:100vh; display:flex; align-items:center;
                justify-content:center; padding:24px; }
 .page-full   { min-height:100vh; padding:28px 32px; }
 .max-w-md    { max-width:460px; width:100%; }
 .max-w-xl    { max-width:980px; width:100%; margin:0 auto; }
```

```
.card { background:var(--surface); border:1px solid var(--border);
      border-radius:var(--radius-lg); padding:32px; box-shadow:var(--shadow); }
.card-sm { padding:20px; }

h1 { font-family:'Syne',sans-serif; font-size:clamp(22px,4vw,30px);
   font-weight:800; line-height:1.2; }
h2 { font-family:'Syne',sans-serif; font-size:20px; font-weight:700; }
h3 { font-family:'Syne',sans-serif; font-size:16px; font-weight:600; }
.text-gradient { background:linear-gradient(135deg,#6366f1,#a78bfa);
            -webkit-background-clip:text; -webkit-text-fill-color:transparent; }
.text-muted { color:var(--muted); font-size:14px; }
.text-sm    { font-size:13px; }
p.text-muted { color:var(--muted); }

.brand-icon { width:56px; height:56px; border-radius:14px; margin:0 auto 16px;
          background:linear-gradient(135deg,#6366f1,#a78bfa);
          display:flex; align-items:center; justify-content:center;
          box-shadow:var(--glow); }
.brand-icon svg { width:28px; height:28px; stroke:#fff; fill:none; }
.brand-icon-sm  { width:36px; height:36px; border-radius:10px; margin:0;
             background:linear-gradient(135deg,#6366f1,#a78bfa);
             display:flex; align-items:center; justify-content:center; }
.brand-icon-sm svg { width:18px; height:18px; stroke:#fff; fill:none; }

.field { display:flex; flex-direction:column; gap:6px; margin-bottom:14px; }
label  { font-size:13px; font-weight:500; color:var(--muted); }
input[type=text], input[type=email], input[type=password],
input[type=tel],  input[type=date],  input[type=search], select {
  background:var(--surface2); border:1px solid var(--border);
  border-radius:var(--radius); color:var(--text); font-family:inherit;
  font-size:15px; padding:10px 14px; width:100%; outline:none;
  transition:border-color .15s, box-shadow .15s;
}
input:focus, select:focus { border-color:var(--primary);
                      box-shadow:0 0 0 3px rgba(99,102,241,.2); }
input::placeholder { color:var(--muted); }

.btn { display:inline-flex; align-items:center; justify-content:center; gap:8px;
     padding:11px 22px; border:none; border-radius:var(--radius); font-family:inherit;
     font-size:15px; font-weight:600; cursor:pointer; text-decoration:none;
     transition:all .18s; width:100%; }
.btn-primary { background:linear-gradient(135deg,#6366f1,#4f46e5); color:#fff;
           box-shadow:0 2px 12px rgba(99,102,241,.35); }
.btn-primary:hover { opacity:.9; transform:translateY(-1px); }
.btn-ghost   { background:var(--surface2); border:1px solid var(--border); color:var(--text); }
.btn-ghost:hover { border-color:var(--primary); }
.btn-danger  { background:#7f1d1d; color:#fca5a5; border:1px solid #991b1b; }
.btn-danger:hover  { background:#991b1b; }
```

```
.btn-success { background:#14532d; color:#86efac; border:1px solid #166534; }
.btn-success:hover { background:#166534; }
.btn-sm  { padding:7px 14px; font-size:13px; width:auto; }
.btn-auto { width:auto; }

.alert { padding:12px 16px; border-radius:var(--radius); font-size:14px;
       margin-bottom:14px; line-height:1.5; }
.alert-error   { background:rgba(239,68,68,.1);  color:#fca5a5; border:1px solid rgba(239,68,68,.3); }
.alert-success { background:rgba(34,197,94,.1);  color:#86efac; border:1px solid rgba(34,197,94,.3);
}
.alert-info    { background:rgba(99,102,241,.1); color:#a5b4fc; border:1px solid rgba(99,102,241,.3);
}
.alert-warn    { background:rgba(245,158,11,.1); color:#fcd34d; border:1px solid
rgba(245,158,11,.3); }

.badge        { display:inline-block; padding:3px 10px; border-radius:20px;
              font-size:12px; font-weight:600; text-transform:uppercase; letter-spacing:.04em; }
.badge-pending, .pending { background:rgba(99,102,241,.15); color:#a5b4fc; }
.badge-success, .success { background:rgba(34,197,94,.15);  color:#86efac; }
.badge-failed,  .failed  { background:rgba(239,68,68,.15);  color:#fca5a5; }

.plans-grid { display:grid; grid-template-columns:repeat(auto-fill,minmax(200px,1fr));
            gap:16px; margin:20px 0; }
.plan-card  { background:var(--surface2); border:1px solid var(--border);
            border-radius:var(--radius-lg); padding:20px; cursor:pointer;
            transition:border-color .2s, transform .2s, box-shadow .2s;
            text-align:left; width:100%; }
.plan-card:hover { border-color:var(--primary); transform:translateY(-3px);
                box-shadow:0 8px 24px rgba(99,102,241,.2); }
.plan-name  { font-family:'Syne',sans-serif; font-size:17px; font-weight:700; margin-bottom:4px; }
.plan-meta  { font-size:13px; color:var(--muted); margin-bottom:10px; }
.plan-price { font-family:'Syne',sans-serif; font-size:26px; font-weight:800; color:var(--primary); }
.plan-price span { font-size:13px; font-weight:400; color:var(--muted); }

.divider { display:flex; align-items:center; gap:12px; margin:18px 0; color:var(--muted);
         font-size:12px; text-transform:uppercase; letter-spacing:.06em; }
.divider::before, .divider::after { content:''; flex:1; height:1px; background:var(--border); }

table { width:100%; border-collapse:collapse; font-size:13px; }
th    { text-align:left; color:var(--muted); font-weight:500; padding:10px 12px;
       border-bottom:1px solid var(--border); white-space:nowrap; }
td    { padding:10px 12px; border-bottom:1px solid rgba(255,255,255,.03); vertical-align:middle; }
tr:hover td { background:rgba(255,255,255,.02); }

.stats-grid { display:grid; grid-template-columns:repeat(auto-fill,minmax(170px,1fr));
            gap:16px; margin-bottom:24px; }
.stat-card  { background:var(--surface2); border:1px solid var(--border);
            border-radius:var(--radius-lg); padding:20px; }
```

```
 .stat-label { font-size:12px; color:var(--muted); text-transform:uppercase;
            letter-spacing:.06em; margin-bottom:6px; }
 .stat-value { font-family:'Syne',sans-serif; font-size:26px; font-weight:800; }
 .stat-sub   { font-size:12px; color:var(--muted); margin-top:2px; }

 .admin-nav { display:flex; align-items:center; justify-content:space-between;
           padding:0 0 20px; margin-bottom:24px; border-bottom:1px solid var(--border); }
 .admin-nav-left { display:flex; align-items:center; gap:12px; }

 .section-head { display:flex; align-items:center; justify-content:space-between;
              margin:0 0 14px; }

 .filter-bar { display:flex; gap:10px; flex-wrap:wrap; margin-bottom:16px; }
 .filter-bar input, .filter-bar select { width:auto; flex:1; min-width:130px; }

 .mono { font-family:monospace; font-size:12px; }
 .mt-2 { margin-top:8px; }
 .mt-4 { margin-top:16px; }
 .mb-4 { margin-bottom:16px; }
 code  { background:var(--surface2); padding:2px 7px; border-radius:5px;
        font-family:monospace; font-size:12px; }
 a     { color:var(--primary); text-decoration:none; }
 a:hover { text-decoration:underline; }

 .spinner { width:48px; height:48px; border-radius:50%; margin:0 auto 20px;
          border:3px solid rgba(99,102,241,.2); border-top-color:var(--primary);
          animation:spin .8s linear infinite; }
 @keyframes spin { to { transform:rotate(360deg); } }

 .success-icon { width:72px; height:72px; border-radius:50%; margin:0 auto 20px;
              background:linear-gradient(135deg,#6366f1,#a78bfa);
              display:flex; align-items:center; justify-content:center;
              animation:pulse 2s ease-in-out infinite; }
 .success-icon svg { width:36px; height:36px; stroke:#fff; fill:none; }
 @keyframes pulse { 0%,100%{box-shadow:0 0 0 0 rgba(99,102,241,.4)}
                 50%{box-shadow:0 0 0 16px rgba(99,102,241,0)} }
</style>
"""

def page(title: str, body: str, wide: bool = False, status: int = 200) -> Response:
  container = "page-full max-w-xl" if wide else "page-center"
  inner_cls  = "max-w-xl" if wide else "max-w-md"

  html = f"""<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,initial-scale=1.0">
```

```
 <title>{title} — WiFi Portal</title>
 {LOVABLE_CSS}
</head>
<body>
 <div class="{container}">
  <div class="{inner_cls}">
   {body}
  </div>
 </div>
</body>
</html>"""
  return Response(html, status=status, mimetype="text/html")

@app.route("/")
def index():
  return redirect("/login?" + request.query_string.decode())

@app.route("/guest/s/default/")
def unifi_entry():
  return redirect("/login?" + request.query_string.decode())

@app.route("/login")
def login():
  log.info(f"[Login] args={dict(request.args)}")
  print("ARGS:", request.args)
  mac = request.args.get("mac") or request.args.get("id")

  if not mac:
    log.warning("[Login] No MAC provided")
    return page("Connect", """
      <div class="card" style="text-align:center">
       <div class="brand-icon">
        <svg viewBox="0 0 24 24" stroke-width="2">
         <path d="M5 12.55a11 11 0 0 1 14.08 0" stroke-linecap="round"/>
         <path d="M1.42 9a16 16 0 0 1 21.16 0" stroke-linecap="round"/>
         <path d="M8.53 16.11a6 6 0 0 1 6.95 0" stroke-linecap="round"/>
         <circle cx="12" cy="20" r="1" fill="white" stroke="white"/>
        </svg>
       </div>
       <h1>WiFi Portal</h1>
       <p class="text-muted mt-2">
        No device detected. Please reconnect to the WiFi network and try again.
       </p>
      </div>""", status=400)

  mac = mac.strip().lower()
  session["mac"]        = mac
  session["redirect_url"] = request.args.get("url", "")
```

```
  log.info(f"[Login] MAC captured: {mac}")

  query(
    """
    INSERT INTO portal_sessions (mac_address, created_at)
    VALUES (%s, NOW())
    ON CONFLICT (mac_address) DO UPDATE SET created_at = NOW()
    """,
    (mac,), commit=True,
  )

  return redirect(url_for("plans"))

@app.route("/plans")
def plans():
  mac = session.get("mac")
  if not mac:
    return redirect(url_for("login"))

  rows = query("SELECT * FROM plans ORDER BY price_kes ASC", fetchall=True)

  if not rows:
    return page("Plans", """
      <div class="card" style="text-align:center">
       <h2>No Plans Available</h2>
       <p class="text-muted mt-2">Please contact venue staff.</p>
      </div>""")

  cards_html = ""
  for p in rows:
    dur = p["duration_minutes"]
    dur_label = (
      f"{dur} min" if dur < 60
      else f"{dur // 60} hr" if dur < 1440
      else f"{dur // 1440} day{'s' if dur // 1440 > 1 else ''}"
    )
    speed = f"{p['bandwidth_mbps']} Mbps" if p.get("bandwidth_mbps") else "Unlimited speed"
    data  = f"{p['data_cap_mb']} MB"     if p.get("data_cap_mb")    else "Unlimited data"
    desc  = p.get("description") or ""

    cards_html += f"""
    <form method="POST" action="/select-plan">
     <input type="hidden" name="plan_id" value="{p['id']}">
     <button type="submit" class="plan-card">
      <div class="plan-name">{p['name']}</div>
      <div class="plan-meta">{dur_label}  ·  {speed}  ·  {data}</div>
      {'<div class="plan-meta">' + desc + '</div>' if desc else ''}
```

```
        <div class="plan-price">KES {p['price_kes']:,} <span>/ session</span></div>
       </button>
      </form>"""

  body = f"""
   <div style="text-align:center; margin-bottom:28px">
    <div class="brand-icon">
     <svg viewBox="0 0 24 24" stroke-width="2">
      <path d="M5 12.55a11 11 0 0 1 14.08 0" stroke-linecap="round"/>
      <path d="M1.42 9a16 16 0 0 1 21.16 0" stroke-linecap="round"/>
      <path d="M8.53 16.11a6 6 0 0 1 6.95 0" stroke-linecap="round"/>
      <circle cx="12" cy="20" r="1" fill="white" stroke="white"/>
     </svg>
    </div>
    <h1>Choose a <span class="text-gradient">WiFi Plan</span></h1>
    <p class="text-muted mt-2" style="margin-top:6px">
     Device: <code>{mac}</code>
    </p>
   </div>
   <div class="plans-grid">{cards_html}</div>"""

  return page("Plans", body)

@app.route("/select-plan", methods=["POST"])
def select_plan():
  mac     = session.get("mac")
  plan_id = request.form.get("plan_id", "").strip()

  if not mac:
    return redirect(url_for("login"))

  plan = query("SELECT * FROM plans WHERE id = %s", (plan_id,), fetchone=True)
  if not plan:
    log.warning(f"[SelectPlan] Invalid plan_id={plan_id} for mac={mac}")
    return page("Error", """
      <div class="card">
       <div class="alert alert-error">Invalid plan selected.</div>
       <a href="/plans" class="btn btn-ghost mt-4">&#8592; Back to Plans</a>
      </div>""", status=400)

  ext_ref = str(uuid.uuid4())

  query(
    """
    INSERT INTO transactions
      (external_reference, mac_address, plan_id, amount, status, created_at)
    VALUES (%s, %s, %s, %s, 'pending', NOW())
    """,
```

```
    (ext_ref, mac, plan_id, plan["price_kes"]),
    commit=True,
  )

  session["external_reference"] = ext_ref
  session["plan_name"]         = plan["name"]
  session["plan_amount"]       = plan["price_kes"]
  session["plan_duration"]     = plan["duration_minutes"]

  log.info(f"[SelectPlan] ref={ext_ref} mac={mac} plan={plan['name']}")
  return redirect(url_for("checkout"))

@app.route("/checkout", methods=["GET", "POST"])
def checkout():
  mac    = session.get("mac")
  ext_ref = session.get("external_reference")

  if not mac or not ext_ref:
    return redirect(url_for("login"))

  plan_name   = session.get("plan_name", "")
  plan_amount = session.get("plan_amount", 0)

  if request.method == "GET":
    body = f"""
     <div class="card">
      <h2 style="margin-bottom:16px">Complete Payment</h2>
      <div class="alert alert-info">
       <strong>{plan_name}</strong>  &mdash;  KES {plan_amount:,}
      </div>
      <form method="POST">
       <div class="field">
        <label>M-Pesa Phone Number</label>
        <input type="tel" name="phone" placeholder="2547XXXXXXXX"
             required maxlength="12" autocomplete="tel">
        <p class="text-sm text-muted mt-2" style="margin-top:6px">
         Format: 2547XXXXXXXX (no + or spaces)
        </p>
       </div>
       <button type="submit" class="btn btn-primary">
        Pay KES {plan_amount:,} via M-Pesa
       </button>
      </form>
      <a href="/plans" class="btn btn-ghost" style="margin-top:10px">&#8592; Change Plan</a>
     </div>"""
    return page("Checkout", body)

  raw   = request.form.get("phone", "").strip()
```

```
  phone = format_phone(raw) or raw

  if not validate_phone(phone):
     log.warning(f"[Checkout] Invalid phone: {phone}")
     body = f"""
      <div class="card">
       <div class="alert alert-error">
        Invalid phone number. Use format 2547XXXXXXXX.
       </div>
       <div class="alert alert-info">
        <strong>{plan_name}</strong>  &mdash;  KES {plan_amount:,}
       </div>
       <form method="POST">
        <div class="field">
         <label>M-Pesa Phone Number</label>
         <input type="tel" name="phone" value="{phone}"
              placeholder="2547XXXXXXXX" required maxlength="12">
         <p class="text-sm text-muted mt-2" style="margin-top:6px">
          Format: 2547XXXXXXXX
         </p>
        </div>
        <button type="submit" class="btn btn-primary">
         Pay KES {plan_amount:,} via M-Pesa
        </button>
       </form>
      </div>"""
     return page("Checkout", body)

  query(
     "UPDATE transactions SET phone_number = %s WHERE external_reference = %s",
     (phone, ext_ref), commit=True,
  )

  ok, err = payhero_stk_push(
     phone=phone,
     amount=plan_amount,
     reference=ext_ref,
     email=phone,
  )

  if not ok:
     log.error(f"[Checkout] STK push failed: {err}")
     query(
        "UPDATE transactions SET status='failed', updated_at=NOW() WHERE
external_reference=%s",
        (ext_ref,), commit=True,
     )
     body = f"""
```

```
      <div class="card">
        <div class="alert alert-error"><strong>Payment Error:</strong> {err}</div>
        <a href="/checkout" class="btn btn-primary">Try Again</a>
        <a href="/plans"    class="btn btn-ghost"   style="margin-top:10px">Change Plan</a>
      </div>"""
    return page("Payment Error", body, status=502)

  log.info(f"[Checkout] STK push sent: ref={ext_ref} phone={phone}")

  body = f"""
    <div class="card" style="text-align:center; padding:40px 32px">
      <div class="spinner" id="spinner"></div>
      <h2>Check Your Phone</h2>
      <p class="text-muted mt-2" style="margin-top:8px">
        An M-Pesa prompt was sent to <strong>{phone}</strong>.<br>
        Enter your PIN to complete the payment.
      </p>
      <div style="margin:24px 0">
        <span class="badge badge-pending" id="status-badge">Waiting for payment&hellip;</span>
      </div>
      <p class="text-sm text-muted" id="status-msg">Checking every 2 seconds&hellip;</p>
      <a href="/checkout" class="btn btn-ghost" id="retry-btn"
         style="display:none; margin-top:16px">&#8635; Try Again</a>
    </div>
    <script>
      var ref = "{ext_ref}";
      var attempts = 0, maxAttempts = 90;
      function poll() {{
        if (++attempts > maxAttempts) {{
          document.getElementById('status-badge').textContent = 'Timed out';
          document.getElementById('status-badge').className   = 'badge badge-failed';
          document.getElementById('status-msg').textContent   = 'Confirmation timed out. Try again.';
          document.getElementById('retry-btn').style.display  = 'inline-flex';
          document.getElementById('spinner').style.display    = 'none';
          return;
        }}
        fetch('/status_api/' + ref)
          .then(function(r) {{ return r.json(); }})
          .then(function(d) {{
            if (d.status === 'success') {{
              document.getElementById('status-badge').textContent = 'Payment confirmed!';
              document.getElementById('status-badge').className   = 'badge badge-success';
              document.getElementById('status-msg').textContent   = 'Redirecting\u2026';
              document.getElementById('spinner').style.borderTopColor = '#22c55e';
              setTimeout(function() {{ window.location = '/success'; }}, 1200);
            }} else if (d.status === 'failed') {{
              document.getElementById('status-badge').textContent = 'Payment failed';
              document.getElementById('status-badge').className   = 'badge badge-failed';
```

```
            document.getElementById('status-msg').textContent   = 'Payment was declined or
cancelled.';
            document.getElementById('retry-btn').style.display  = 'inline-flex';
            document.getElementById('spinner').style.display    = 'none';
          }} else {{
            setTimeout(poll, 2000);
          }}
        }})
        .catch(function() {{ setTimeout(poll, 3000); }});
    }}
    poll();
  </script>"""

  return page("Waiting for Payment", body)

@app.route("/status_api/<external_reference>")
def status_api(external_reference):
  tx = query(
    "SELECT status FROM transactions WHERE external_reference = %s",
    (external_reference,), fetchone=True,
  )
  if not tx:
    return jsonify({"status": "not_found"}), 404
  log.debug(f"[StatusAPI] ref={external_reference[:8]}… → {tx['status']}")
  return jsonify({"status": tx["status"]})

@app.route("/status/<external_reference>")
def status(external_reference):
  tx = query(
    "SELECT status FROM transactions WHERE external_reference = %s",
    (external_reference,), fetchone=True,
  )
  if not tx:
    return jsonify({"status": "not_found"}), 404
  return jsonify({"status": tx["status"]})

@app.route("/success")
def success():
  plan_name = session.get("plan_name", "your plan")
  mac      = session.get("mac")
  duration  = session.get("plan_duration", 60)

  if mac:
    log.info(f"[Success] Re-authorizing mac={mac} for {duration} min")
    authorize_client(mac, duration)

  body = f"""
   <div class="card" style="text-align:center; padding:40px 32px">
```

```
      <div class="success-icon">
        <svg viewBox="0 0 24 24" stroke-width="2.5">
          <circle cx="12" cy="12" r="10"/>
          <path d="m9 12 2 2 4-4" stroke-linecap="round" stroke-linejoin="round"/>
        </svg>
      </div>
      <h1>You're <span class="text-gradient">Connected!</span></h1>
      <p class="text-muted" style="margin-top:8px">
        Payment confirmed &mdash; enjoy <strong>{plan_name}</strong>!
      </p>
      <div class="alert alert-success" style="margin-top:20px; text-align:left">
        <strong>&#10003; Internet access granted</strong><br>
        Your device has been authorized on the network.
      </div>
      <a href="/plans" class="btn btn-ghost" style="margin-top:16px">Buy Another Plan</a>
    </div>"""

  return page("Connected!", body)

@app.route("/callback", methods=["POST"])
def callback():
  data = request.get_json(silent=True)

  if not data:
    return jsonify({"error": "Invalid JSON"}), 400

  print("📥 PAYHERO CALLBACK:", data)

  response = data.get("response", {})

  reference = response.get("ExternalReference")
  status = str(response.get("Status", "")).lower()

  if not reference:
    return jsonify({"error": "Missing reference"}), 400

  tx = query(
    "SELECT * FROM transactions WHERE external_reference = %s",
    (reference,),
    fetchone=True,
  )

  if not tx:
    return jsonify({"error": "Transaction not found"}), 404

  if status == "success":
    new_status = "success"
```

```
    plan = query(
        "SELECT * FROM plans WHERE id = %s",
        (tx["plan_id"],),
        fetchone=True
    )

    if not plan:
        print("⚠ Plan not found, defaulting to 60 minutes")
        minutes = 60
    else:
        minutes = plan.get("duration_minutes", 60)

    query(
        """
        INSERT INTO portal_sessions
            (mac_address, transaction_id, plan_id, expires_at, created_at)
        VALUES (%s, %s, %s, NOW() + (%s || ' minutes')::interval, NOW())
        ON CONFLICT (mac_address) DO UPDATE SET
            transaction_id = EXCLUDED.transaction_id,
            plan_id       = EXCLUDED.plan_id,
            expires_at    = EXCLUDED.expires_at,
            created_at    = EXCLUDED.created_at
        """,
        (tx["mac_address"], tx["id"], tx["plan_id"], str(minutes)),
        commit=True,
    )

    unifi_ok = authorize_client(tx["mac_address"], minutes)

    if unifi_ok:
        log.info(f"[Callback] UniFi authorized: mac={tx['mac_address']}")
    else:
        log.error(f"[Callback] UniFi FAILED: mac={tx['mac_address']}")

elif status == "failed":
    new_status = "failed"
else:
    new_status = "pending"

query(
    "UPDATE transactions SET status=%s WHERE id=%s",
    (new_status, tx["id"]),
    commit=True
)

return jsonify({"ok": True}), 200

@app.route("/admin/login", methods=["GET", "POST"])
```

```
def admin_login():
  if request.method == "GET":
     body = """
      <div class="card" style="max-width:360px; margin:0 auto">
       <div style="text-align:center; margin-bottom:24px">
        <div class="brand-icon">
         <svg viewBox="0 0 24 24" stroke-width="2">
          <rect x="3" y="11" width="18" height="11" rx="2" ry="2"/>
          <path d="M7 11V7a5 5 0 0 1 10 0v4"/>
         </svg>
        </div>
        <h2>Admin Access</h2>
        <p class="text-muted" style="margin-top:4px">WiFi Portal Dashboard</p>
       </div>
       <form method="POST" action="/admin/login">
        <div class="field">
         <label>Admin Password</label>
         <input type="password" name="password"
              placeholder="Enter admin password" required autofocus>
        </div>
        <button type="submit" class="btn btn-primary">Sign In</button>
       </form>
      </div>"""
     return page("Admin Login", body)

  pwd = request.form.get("password", "")
  if pwd == ADMIN_PASSWORD:
     session["admin"] = True
     log.info("[Admin] Successful login")
     return redirect(url_for("admin_dashboard"))

  log.warning("[Admin] Failed login attempt")
  body = """
   <div class="card" style="max-width:360px; margin:0 auto">
     <div style="text-align:center; margin-bottom:24px">
      <div class="brand-icon">
       <svg viewBox="0 0 24 24" stroke-width="2">
        <rect x="3" y="11" width="18" height="11" rx="2" ry="2"/>
        <path d="M7 11V7a5 5 0 0 1 10 0v4"/>
       </svg>
      </div>
      <h2>Admin Access</h2>
     </div>
     <div class="alert alert-error">Incorrect password.</div>
     <form method="POST" action="/admin/login">
      <div class="field">
       <label>Admin Password</label>
       <input type="password" name="password" placeholder="Try again" required autofocus>
```

```
      </div>
      <button type="submit" class="btn btn-primary">Sign In</button>
    </form>
   </div>"""
  return page("Admin Login", body)

@app.route("/admin/logout")
def admin_logout():
  session.pop("admin", None)
  return redirect(url_for("admin_login"))

@app.route("/admin/dashboard")
@login_required_admin
def admin_dashboard():
  search_phone  = request.args.get("phone",  "").strip()
  filter_status = request.args.get("status", "").strip()
  filter_date   = request.args.get("date",   "").strip()

  total_users = (query(
    "SELECT COUNT(DISTINCT mac_address) AS c FROM transactions",
    fetchone=True) or {}).get("c", 0)

  active_sessions = (query(
    "SELECT COUNT(*) AS c FROM portal_sessions WHERE expires_at > NOW()",
    fetchone=True) or {}).get("c", 0)

  total_revenue = int((query(
    "SELECT COALESCE(SUM(amount),0) AS s FROM transactions WHERE status='success'",
    fetchone=True) or {}).get("s", 0))

  today_revenue = int((query(
    "SELECT COALESCE(SUM(amount),0) AS s FROM transactions "
    "WHERE status='success' AND created_at::date = CURRENT_DATE",
    fetchone=True) or {}).get("s", 0))

  pending_count = (query(
    "SELECT COUNT(*) AS c FROM transactions WHERE status='pending'",
    fetchone=True) or {}).get("c", 0)

  active_users = query(
    """
    SELECT ps.mac_address, ps.created_at, ps.expires_at,
         p.name AS plan_name, p.duration_minutes,
         GREATEST(0, EXTRACT(EPOCH FROM (ps.expires_at - NOW()))/60)::int AS mins_left
    FROM portal_sessions ps
    LEFT JOIN plans p ON p.id = ps.plan_id
    WHERE ps.expires_at > NOW()
    ORDER BY ps.expires_at ASC
```

```
    """,
    fetchall=True,
  ) or []

  tx_sql    = """
    SELECT t.external_reference, t.mac_address, t.phone_number,
         t.amount, t.status, t.created_at, t.updated_at,
         p.name AS plan_name
    FROM transactions t
    LEFT JOIN plans p ON p.id = t.plan_id
    WHERE 1=1
  """
  tx_params = []

  if search_phone:
    tx_sql += " AND t.phone_number ILIKE %s"
    tx_params.append(f"%{search_phone}%")
  if filter_status:
    tx_sql += " AND t.status = %s"
    tx_params.append(filter_status)
  if filter_date:
    tx_sql += " AND t.created_at::date = %s"
    tx_params.append(filter_date)

  tx_sql += " ORDER BY t.created_at DESC LIMIT 100"
  transactions = query(tx_sql, tx_params, fetchall=True) or []

  metrics_html = f"""
  <div class="stats-grid">
   <div class="stat-card">
    <div class="stat-label">Total Devices</div>
    <div class="stat-value">{total_users}</div>
    <div class="stat-sub">Unique MACs</div>
   </div>
   <div class="stat-card">
    <div class="stat-label">Active Sessions</div>
    <div class="stat-value" style="color:var(--success)">{active_sessions}</div>
    <div class="stat-sub">Currently online</div>
   </div>
   <div class="stat-card">
    <div class="stat-label">Total Revenue</div>
    <div class="stat-value">KES {total_revenue:,}</div>
    <div class="stat-sub">All time</div>
   </div>
   <div class="stat-card">
    <div class="stat-label">Today's Revenue</div>
    <div class="stat-value" style="color:var(--primary)">KES {today_revenue:,}</div>
    <div class="stat-sub">Since midnight UTC</div>
```

```
  </div>
  <div class="stat-card">
   <div class="stat-label">Pending Payments</div>
   <div class="stat-value" style="color:var(--warning)">{pending_count}</div>
   <div class="stat-sub">Awaiting confirmation</div>
  </div>
</div>"""

active_rows = ""
for u in active_users:
    mac  = u["mac_address"]
    plan = u.get("plan_name") or "—"
    left = u.get("mins_left", 0)
    exp  = str(u.get("expires_at", ""))[:16]
    active_rows += f"""
    <tr>
     <td><code>{mac}</code></td>
     <td>{plan}</td>
     <td>{exp}</td>
     <td><span class="badge badge-success">{left} min left</span></td>
     <td style="white-space:nowrap">
      <form method="POST" action="/admin/reauthorize" style="display:inline">
       <input type="hidden" name="mac" value="{mac}">
       <button type="submit" class="btn btn-success btn-sm">Re-auth</button>
      </form>
       
      <form method="POST" action="/admin/disconnect" style="display:inline">
       <input type="hidden" name="mac" value="{mac}">
       <button type="submit" class="btn btn-danger btn-sm">Disconnect</button>
      </form>
     </td>
    </tr>"""

no_active = '<p class="text-muted">No active sessions right now.</p>'
active_body = f"""
    <div style="overflow-x:auto">
     <table>
      <thead><tr>
       <th>MAC Address</th><th>Plan</th><th>Expires</th>
       <th>Time Left</th><th>Actions</th>
      </tr></thead>
      <tbody>{active_rows}</tbody>
     </table>
    </div>""" if active_rows else no_active

active_section = f"""
<div class="card mb-4" style="margin-bottom:20px">
 <div class="section-head">
```

```
    <h3>Active WiFi Users ({len(active_users)})</h3>
   </div>
   {active_body}
  </div>"""

  tx_rows = ""
  for tx in transactions:
     cls = {
        "success": "badge-success",
        "failed":  "badge-failed",
     }.get(tx["status"], "badge-pending")
     ts  = str(tx.get("created_at", ""))[:16]
     ref = str(tx.get("external_reference", ""))
     ref_short = ref[:12] + "…"
     tx_rows += f"""
     <tr>
      <td><code title="{ref}">{ref_short}</code></td>
      <td><code>{tx.get('mac_address','—')}</code></td>
      <td>{tx.get('phone_number','—')}</td>
      <td>{tx.get('plan_name','—')}</td>
      <td>KES {tx.get('amount',0):,}</td>
      <td><span class="badge {cls}">{tx['status']}</span></td>
      <td style="color:var(--muted)">{ts}</td>
     </tr>"""

  no_tx = '<p class="text-muted">No transactions match the current filters.</p>'
  tx_body = f"""
     <div style="overflow-x:auto">
      <table>
       <thead><tr>
        <th>Reference</th><th>MAC</th><th>Phone</th><th>Plan</th>
        <th>Amount</th><th>Status</th><th>Date</th>
       </tr></thead>
       <tbody>{tx_rows}</tbody>
      </table>
     </div>""" if tx_rows else no_tx

  tx_section = f"""
  <div class="card">
   <div class="section-head">
    <h3>Transactions</h3>
    <span class="text-muted text-sm">{len(transactions)} shown</span>
   </div>
   <form method="GET" class="filter-bar">
    <input type="text"  name="phone"  placeholder="Search phone&hellip;"
value="{search_phone}">
    <select name="status">
     <option value="">All statuses</option>
```

```
      <option value="pending" {'selected' if filter_status=='pending' else ''}>Pending</option>
      <option value="success" {'selected' if filter_status=='success' else ''}>Success</option>
      <option value="failed"  {'selected' if filter_status=='failed'  else ''}>Failed</option>
    </select>
    <input type="date" name="date" value="{filter_date}">
    <button type="submit" class="btn btn-ghost btn-sm btn-auto">Filter</button>
    <a href="/admin/dashboard" class="btn btn-ghost btn-sm btn-auto">Clear</a>
  </form>
  {tx_body}
</div>"""

  nav = """
  <div class="admin-nav">
   <div class="admin-nav-left">
    <div class="brand-icon-sm">
     <svg viewBox="0 0 24 24" stroke-width="2">
      <rect x="3" y="11" width="18" height="11" rx="2"/>
      <path d="M7 11V7a5 5 0 0 1 10 0v4"/>
     </svg>
    </div>
    <h2>Admin Dashboard</h2>
   </div>
   <a href="/admin/logout" class="btn btn-danger btn-sm btn-auto">Sign Out</a>
  </div>"""

  body = nav + metrics_html + active_section + tx_section
  return page("Admin Dashboard", body, wide=True)

@app.route("/admin/reauthorize", methods=["POST"])
@login_required_admin
def admin_reauthorize():
  mac = request.form.get("mac", "").strip().lower()
  if not mac:
     return redirect(url_for("admin_dashboard"))

  row = query(
     """
     SELECT p.duration_minutes
     FROM portal_sessions ps
     LEFT JOIN plans p ON p.id = ps.plan_id
     WHERE ps.mac_address = %s
     """,
     (mac,), fetchone=True,
  )
  minutes = int(row["duration_minutes"]) if row and row.get("duration_minutes") else 60

  ok = authorize_client(mac, minutes)
  log.info(f"[Admin] Re-authorize mac={mac} minutes={minutes} ok={ok}")
```

```
  if ok:
    query(
      """
      UPDATE portal_sessions
        SET expires_at = NOW() + (%s || ' minutes')::interval
       WHERE mac_address = %s
      """,
      (str(minutes), mac), commit=True,
    )

  return redirect(url_for("admin_dashboard"))

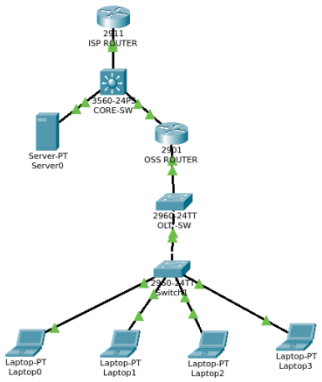
@app.route("/admin/disconnect", methods=["POST"])
@login_required_admin
def admin_disconnect():
  mac = request.form.get("mac", "").strip().lower()
  if not mac:
    return redirect(url_for("admin_dashboard"))

  ok = disconnect_client(mac)
  log.info(f"[Admin] Disconnect mac={mac} ok={ok}")

  if ok:
    query(
      "UPDATE portal_sessions SET expires_at = NOW() WHERE mac_address = %s",
      (mac,), commit=True,
    )

  return redirect(url_for("admin_dashboard"))
```

Appendix D: Network Topology Diagrams



IMPLEMENTED TOPOLOGY

PHYSICAL WIRED TOPOLOGY



Appendix E: Testing Logs and Results [Sample ping tests, API response logs, transaction records]



MY BILLING SYSTEM PROTOTYPE WEBISTE

The University of Nairobi
F17/142583/2021



SHOWCASE WEBSITE FOR INTERNET PROTOCOLS AND DEMOS

The University of Nairobi
F17/142583/2021